
Zeroth-Order Forward-Only SNN Training Inspiring Neuromorphic On-Chip Learning

Mingyue Qin^{*1} Shuyu Yin^{*1} Qinghai Guo² Peilin Liu¹ Xiaolin Huang¹ Fei Wen¹

Abstract

The human brain is a biologically instantiated on-device neural system that integrates both learning and inference in a unified architecture, which enables rapid and flexible learning on-the-fly. This extraordinary online learning ability is realized through biological learning mechanisms operating on a well-initialized innate model. This work considers the on-chip edge learning upon pretrained models with zeroth-order (ZO) methods. ZO optimization methods, which resemble bio-plausible perturbation-based learning, offer a promising approach that enables learning with only forward passes and hence can significantly reduce the complexity of on-chip hardware implementation. However, in this work we show that applying ZO methods to spiking neural networks (SNNs) is non-trivial due to the step-function nature of spiking activation. We analyze the challenges posed by the spiking activation, and reveal a variance amplification effect of it. Based on this insight, we propose a subspace-based ZO (SZO) method that leverages the intrinsic low-dimensional structure of the SNN optimization trajectory. By learning in a low-dimensional subspace, SZO substantially enhances ZO learning efficacy, achieving accuracy comparable to first-order (FO) methods with faster learning speed than full-space BP. We evaluate SZO on model training from scratch, continual training, and unsupervised adaptation. Experimental results demonstrate that SZO closely approaches FO training performance for the first time while offering fast learning speed. Code is available at <https://github.com/LeviD536/SZO>.

1. Introduction

For edge AI applications, two brain-like capabilities are particularly desirable: 1) *High energy efficiency*. 2) *Extraordinary online learning ability*. Functionally, the brain can be regarded as a biologically instantiated on-device neural system that integrates both learning and inference within a unified architecture, which enables flexible and adaptive learning on-the-fly. Realizing these capabilities in neuromorphic computing systems to achieve efficient on-chip learning for edge devices has great promise (Christensen et al., 2022), which has inspired many researches on spiking neural networks (SNNs) (Maass, 1997; Zenke & Ganguli, 2018; Lee et al., 2016; Neftci et al., 2019; Fang et al., 2021), and neuromorphic hardware that support on-chip learning, such as BrainScaleS (Schemmel et al., 2010) and Loihi (Davies et al., 2018). A key challenge lies in designing highly efficient SNN learning algorithms with low hardware implementation complexity.

Recent years have witnessed rapid progress in SNN algorithms that successfully narrows the performance gap with ANNs (Meng et al., 2023; Hu et al., 2021; Li et al., 2021; Yin et al., 2021; Xiao et al., 2022; Fang et al., 2023b; Zhou et al., 2023; Yao et al., 2024; Zhou et al., 2024). Most of these advances rely on BPTT. However, BPTT requires unfolding the backward computational graph across time-steps, which incurs substantial computational and memory overhead that makes it unsuitable for resource-constrained edge devices. To mitigate this, approximate BPTT algorithms using only forward-in-time computation are proposed (Bellec et al., 2020; Xiao et al., 2022; Meng et al., 2023; Yin et al., 2023). Nonetheless, these algorithms still rely on spatial BP, which remains too demanding for practical on-chip implementation, especially for modest and large models.

Specifically, BP is not only biologically implausible (Lillicrap et al., 2020) but also too complex to implement on resource-constrained edge hardware: **(1) the need for dedicated feedback circuits** for error backpropagation, **(2) the update locking problem**, where forward computation must wait for backward computation, **(3) high memory overhead** from storing activations, especially as batch-size and time-steps increase, **(4) inefficient weight transport**, involving hardware-inefficient weight transfer during backward

^{*}Equal contribution ¹Shanghai Jiao Tong University

²Huawei Technologies. Correspondence to: Fei Wen <wen-fei@sjtu.edu.cn>.

passes. In comparison, local learning approaches (Kaiser et al., 2020) and Hebbian learning methods are highly parallelizable in hardware and have been successfully implemented on neuromorphic platforms (Furber et al., 2014; Davies et al., 2018; Baek et al., 2019; Frenkel et al., 2018; 2019; Heidarpur et al., 2019). Nevertheless, these methods lack scalability to deep models and complex tasks.

Zeroth-order (ZO) optimization, which only requires forward computations, offers an attractive alternative for neuromorphic hardware due to its simplicity. Recent studies have shown its effectiveness for fine-tuning LLMs (Mal-ladi et al., 2023; Zhang et al., 2024b). From a bio-inspired perspective, perturbation-based learning, a long-studied biologically grounded approach (Fiete et al., 2007; Bouvier et al., 2018; Aronov et al., 2008; Ali et al., 2013), is an instantiation of ZO optimization. Since it avoids backward gradient computation, it is well suited for hardware implementation without feedback paths and holds great promise for efficient neuromorphic on-chip learning.

However, despite this potential, we show that applying ZO methods to train SNNs is non-trivial due to the step-function nature of the spiking activation (e.g., Heaviside function). Specifically, the step function leads to high variance in ZO gradient estimation, which makes ZO methods ineffective for SNN training. To address this, we propose a subspace-based ZO (SZO) learning algorithm that leverages the intrinsic low-dimensional structure of SNN optimization trajectory to achieve effective SNNs training using forward-only passes. The main contributions are summarized as follows.

- We show that the step-function activation in SNNs introduces high variance in ZO gradient estimation, which substantially degrades the effectiveness of ZO methods.
- We propose a subspace-based zeroth-order (SZO) learning algorithm, which can achieve accuracy comparable to first-order (FO) methods with faster learning speed than vanilla (full-space) BP, which is particularly desirable for on-chip learning scenarios.
- We provide theoretical analysis to show that the proposed method can improve the convergence rate of ZO training for SNN.
- We conduct extensive evaluations of SZO across three learning scenarios, including training from scratch, continual training, and unsupervised adaptation, which demonstrate that SZO achieves state-of-the-art accuracy on these tasks, closely approaching the performance of FO training.

To the best of our knowledge, SZO is the first forward-passes-only SNN training method that can scale to large-scale datasets such as ImageNet. In typical on-device learn-

ing settings, training is performed upon pretrained models. This setup echoes the brain’s learning paradigm (Lillcrap et al., 2020), where neural architectures and synapse strengths are innately initialized and only modest learning is required to acquire new tasks. Therefore, our method offers a promising step toward enabling efficient, scalable neuromorphic on-chip learning for real-world applications.

It is worth noting that Mukhoty et al. (2023) employs local ZO differentiation at the neuron level to address SNN non-differentiability, but still depends on global BP. Moreover, Xiao et al. (2026) proposes a pseudo-ZO algorithm for SNN based on node perturbation in forward computation, yet it requires error feedback to each layer by feedback connections, which resembles direct feedback alignment (Nøkland, 2016). A detailed related work is provided in Appendix A.

2. Preliminaries

2.1. Spiking Neural Networks

Inspired by the brain, SNNs operate in a spike-based processing paradigm that transmits information via temporally sparse binary spikes. This enables energy-efficient processing well-suited to neuromorphic hardware. A widely used neuron model in SNNs is the leaky integrate-and-fire (LIF) model (Burkitt, 2006), whose discrete-time dynamics can be expressed as

$$u_l[t] = \left(1 - \frac{1}{\tau}\right) (u_l[t-1] - V_{th}s_l[t-1]) + W_l s_{l-1}[t] + b_l, \quad (1)$$

$$s_l[t] = H(u_l[t] - V_{th}) = \begin{cases} 1, & u_l[t] > V_{th} \\ 0, & \text{otherwise} \end{cases}, \quad (2)$$

where $u_l[t]$ and $s_l[t]$ denote the membrane potential and output spike vector at time step t for layer l , respectively. W_l and b_l are the corresponding synaptic weights and bias. V_{th} is the spike emission threshold. A spike is emitted when the membrane potential exceeds V_{th} , after which the potential is reset by subtracting V_{th} . $H(\cdot)$ is the Heaviside activation function, which is non-differentiable and poses challenges for gradient-based training. To address this, surrogate gradient methods have been widely used to enable effective training of SNNs via BPTT (Neftci et al., 2019).

2.2. Zeroth-Order Optimization

ZO optimization refers to a class of optimization methods that rely solely on function evaluations without requiring access to explicit gradient information. These methods are particularly well-suited for scenarios where the objective function is non-differentiable or its gradient is inaccessible or costly to compute. Two widely used ZO gradient estimation approaches are coordinate-wise gradient estimation (CGE) and randomized vector-wise gradient estimation

(RGE), defined as

$$\hat{\nabla}^{\text{CGE}} \mathcal{L}_{\mathcal{B}}(\theta) = \sum_{i=1}^d \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu e_i) - \mathcal{L}_{\mathcal{B}}(\theta - \mu e_i)}{2\mu} e_i, \quad (3)$$

$$\hat{\nabla}^{\text{RGE}} \mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{q} \sum_{i=1}^q \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu z_i) - \mathcal{L}_{\mathcal{B}}(\theta - \mu z_i)}{2\mu} z_i, \quad (4)$$

where $\theta \in \mathbb{R}^d$ denotes the model parameters to be optimized, and $z_i \sim \mathcal{N}(0, I_d)$. The function $\mathcal{L}_{\mathcal{B}}(\theta) := \frac{1}{B} \sum_{i=1}^B \mathcal{L}_{x_i}(\theta)$ is the empirical loss evaluated over a mini-batch $\mathcal{B} = \{x_1, \dots, x_B\}$ of size B , with each x_i sampled independently from the distribution $\mathcal{D}$. Here, $\mathcal{L}_x(\theta)$ is the loss evaluated at instance x , and the expected loss is defined as $\mathcal{L}(\theta) = \mathbb{E}_x[\mathcal{L}_x(\theta)] = \mathbb{E}_{\mathcal{B}}[\mathcal{L}_{\mathcal{B}}(\theta)]$. $\mu > 0$ controls the magnitude of parameter perturbations. In CGE, $e_i \in \mathbb{R}^d$ corresponds to the i -th standard basis vector, where all entries are zero except the i -th component equal to 1. For RGE, $z_i \in \mathbb{R}^d$ is a random vector sampled from isotropic distributions, e.g., standard Gaussian $\mathcal{N}(0, I)$ or uniform distribution. The computational complexity of CGE scales linearly with d due to its coordinate-wise operations. In contrast, RGE uses q random perturbations for each gradient estimation. CGE can achieve much better performance than RGE when optimizing deep neural models (Chen et al., 2024).

3. Analysis on the Variance Amplification of Heaviside Activation

While ZO optimization has recently demonstrated success in training ANNs (Chen et al., 2024) and fine-tuning LLMs (Malladi et al., 2023), it faces significant challenges when training SNNs. Figure 1a compares the performance of ZO training of ANN and SNN on CIFAR10 using the same ResNet20 architecture (Chen et al., 2024). The SNN variant is derived by replacing ReLU activations in the ANN ResNet20 with LIF neuron (1), with a time-step of 1. Both networks are trained for 40 epochs using the DeepZero method with its default hyperparameter setting, which employs CGE (3) and has demonstrated success in training deep ANNs while RGE (4) fails (Chen et al., 2024). The results in Figure 1a, together with additional intensive experiments, reveal that ZO methods fail to train SNN. In contrast, DeepZero with CGE performs well on ANN. This highlights a limitation of applying ZO methods directly to SNNs.

To understand the underlying cause of this limitation, we analyze the variance of ZO gradient estimation in ANN and SNN training. Given estimated gradients $G = [\hat{g}_1, \hat{g}_2, \dots, \hat{g}_K] \in \mathbb{R}^{d \times K}$ across K mini-batches with mean gradient $\bar{g} = \frac{1}{K} \sum_{k=1}^K \hat{g}_k$, we compute the variance as $\text{Var}(\hat{g}) = \frac{1}{K} \sum_{k=1}^K \|\hat{g}_k - \bar{g}\|^2$. The left sub-figure in Fig-

ure 1b compares the gradient estimation variance across different layers of the ANN and SNN. It can be seen that, in the early training stage, the variance of the shallow and middle layers of the SNN is much higher than that of the ANN. Moreover, the right sub-figure in Figure 1b compares the proportion of effective perturbations. A perturbation e_i is considered effective if $\mathcal{L}_{\mathcal{B}}(\theta + \mu e_i) \neq \mathcal{L}_{\mathcal{B}}(\theta)$. Clearly, the SNN has a much lower ratio of effective perturbations compared to the ANN.

To further probe the origin of the high variance of SNN, we compare the variance induced by different activation functions, including ReLU, Sigmoid, and Heaviside.

Proposition 3.1 (Variance comparison between activate functions). *Let z be a random variable with variance σ^2 and satisfying that z is symmetric about the origin, i.e. $\Pr(z > t) = \Pr(z < -t)$, and z has a continuous density in a neighbourhood of 0 such that $\Pr(z = 0) = 0$. Define $f_h(z) := \mathbb{I}(z > 0)$, $f_s(z) := \frac{1}{1+e^{-z}}$, and $f_r(z) := \max\{0, z\}$. Then there exists a sufficient small $\sigma_0 > 0$ such that for any $\sigma \in (0, \sigma_0)$ there holds: $\text{Var}(f_h(z)) > \text{Var}(f_s(z))$ and $\text{Var}(f_h(z)) > \text{Var}(f_r(z))$.*

This result indicates that the Heaviside activation introduces significantly higher output variance for a small input perturbation. For a Gaussian example $z \sim \mathcal{N}(0, \sigma^2)$, we have $\text{Var}(f_h(z)) = 0.25$, $\text{Var}(f_s(z)) \approx 0.0625\sigma^2$, and $\text{Var}(f_r(z)) \approx 0.341\sigma^2$. Thus, for a small σ we have $\text{Var}(f_h(z)) \gg \text{Var}(f_r(z)) > \text{Var}(f_s(z))$. In SNNs, this high variance leads to unstable training dynamics and rapid ill-conditioning after only a few epochs, as shown in Figure 1b. Simultaneously, the proportion of effective perturbations diminishes drastically, which leads to training failure. These findings motivate the use of specialized variance reduction approaches in ZO optimization for SNNs.

4. The Proposed SZO Method

4.1. Low-Dimensional Structure in SNN Training

It has been well shown that the training trajectory of ANNs can be approximately covered by a small k -dimension space with $k \ll d$ (Li et al., 2018; Larsen et al., 2022; Gur-Ari et al., 2018; Gressmann et al., 2020; Li et al., 2022). Here we examine this property on SNNs to verify that the low-dimension trajectory hypothesis is also true for SNNs. We conduct experiments using Spike-ResNet18 (11.22M) and Spike-ResNet50 (25.56M) on CIFAR100 and ImageNet1K, respectively. The training is performed using BPTT with surrogate gradient. We set the time-step to 4. This training achieves a test accuracy of 74.42% on CIFAR100 and 64.36% on ImageNet1K.

In the training procedure, we sample $N = 150$ model weights $\{\theta_1, \theta_2, \dots, \theta_N\}$ after each epoch and perform cen-

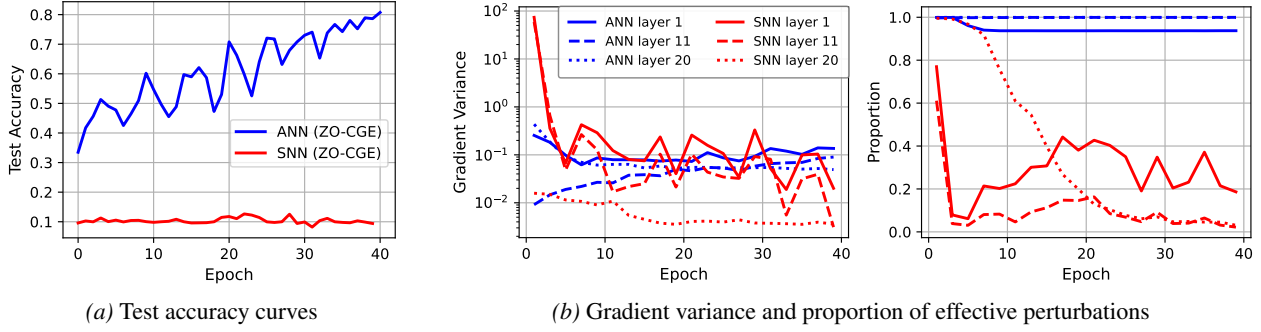
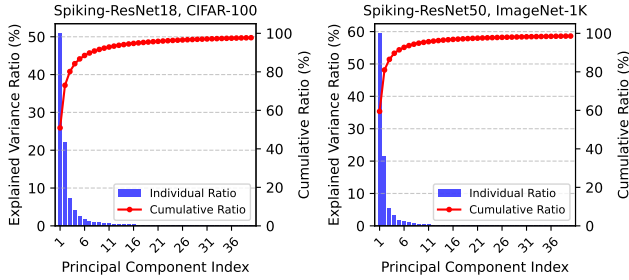


Figure 1. Comparison of ZO training of ANN and SNN using DeepZero with CGE.


 Figure 2. Explained variance analysis of the sampled weights Θ .

tered PCA. Specifically, the sampled weights are first centralized as $\Theta = [\theta_1 - \bar{\theta}, \theta_1 - \bar{\theta}, \dots, \theta_N - \bar{\theta}]$, where $\bar{\theta} = \frac{1}{N} \sum_{i=1}^N \theta_i$. Then, we conduct spectral decomposition on $\Theta\Theta^T$ and obtain k eigenvectors $P = [v_1, v_2, \dots, v_k]$ corresponding to the top- k largest eigenvalues. As depicted in Figure 2, The first ten components account for more than 90% of the total variance for both Spike-ResNet18 on CIFAR100 and Spike-ResNet50 on ImageNet1K. This indicates the existence of a low-dimensional principal subspace that can cover the dominant dynamics of training.

Moreover, as shown in Figure 3 in experiments, training within a 60-dimension subspace spanned by the top eigenvectors P can yield test accuracy comparable to full parameter space training. These results confirm that SNN training trajectories also exhibit a low-dimension structure, and deep SNNs can be efficiently trained in a low-dimension principal subspace. Notably, as shown in Figure 3, subspace-based training converges significantly faster than full-space BP. A theoretical justification of the improved convergence rate of subspace-based learning is provided in Section 5.

4.2. Subspace-Based zeroth-order Optimization for SNN

The variance of ZO gradient estimation scales linearly with the parameter dimension d , which hinders its applicability to deep networks. For example, the RGE (4) has a variance in the order $O(d/q)$ (Duchi et al., 2015), where q is the query number. In SNNs this problem is further aggravated by the variance amplification effect of Heaviside activation,

as shown in Section 3. To address this challenge, we exploit the low-dimension structure of SNN optimization to substantially reduce the variance of ZO gradient estimation and improve the performance of ZO-based SNN training.

Specifically, given the subspace projection matrix $P = [v_1, v_2, \dots, v_k] \in \mathbb{R}^{d \times k}$ with $k \ll d$ as described in Section 4, we optimize an SNN in the subspace spanned by P by SZO-CGE or SZO-RGE as

$$\hat{\nabla}^{\text{SZO-CGE}} \mathcal{L}_{\mathcal{B}}(\theta) = \sum_{i=1}^k \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu v_i) - \mathcal{L}_{\mathcal{B}}(\theta - \mu v_i)}{2\mu} v_i, \quad (5)$$

$$\hat{\nabla}^{\text{SZO-RGE}} \mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{q} \sum_{i=1}^q \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu z_i^P) - \mathcal{L}_{\mathcal{B}}(\theta - \mu z_i^P)}{2\mu} z_i^P,$$

where $z_i^P = Pz_i$ and $z_i \sim \mathcal{N}(0, I_k)$. The parameter update rule with momentum is given by

$$\theta_{t+1} = \theta_t - \eta \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) + \beta(\theta_t - \theta_{t-1}), \quad (6)$$

where η is the learning rate and $\beta \in [0, 1)$ is the momentum coefficient. SZO-CGE involves $2k$ forward passes to estimate the gradient. In experiments, k is typically on the order of a few tens of dimensions, which is much smaller than the parameter dimension d , i.e., $k \ll d$. So SZO-CGE can retain the advantage of CGE but without incurring significant computational overhead. We also propose SZO-RGE, which uses q random perturbations sampled within the subspace to estimate the gradient. According to our experiments, SZO-RGE can further reduce forward passes while achieving comparable accuracy in most cases. Detailed sketch of SZO is summarized in Appendix C.

In this work, we employ PCA to obtain subspace, as it provides a principled method to rank and select basis vectors according to their importance, thereby reducing the number of bases and minimizing memory requirements. While other orthonormalization techniques, such as the Gram-Schmidt process, could also be used, the specific choice of basis construction method is not central to our work. The core of our idea is to construct the subspace from the model’s training trajectory. Thus, the fundamental contribution remains

unchanged regardless of the specific tool used to form the orthonormal basis.

5. Theoretical Analysis

We show that subspace-based ZO training accelerates convergence in SNN compared to full-space ZO training. Since the SNN loss function is non-continuous, we only assume it is bounded and analyze the convergence rate of its smoothed approximation. We first present the some assumptions.

Assumption 5.1 (Boundedness of the original loss). We assume for any $\theta \in \Omega$ and any data $x \sim \mathcal{D}$, the loss $\mathcal{L}_x(\theta)$ is bounded with a constant $0 < M < \infty$ as $|\mathcal{L}_x(\theta)| \leq M$.

In addition, the smoothed version of the original loss with the same perturbation scale as SZO method is defined as

$$\mathcal{L}_x^\mu(\theta) = \mathbb{E}_{u \sim \mathcal{N}(0, I_d)} [\mathcal{L}_x(\theta + \mu u)],$$

where $\mathcal{N}(0, I_d)$ is the normal distribution and I_d is the identity matrix with dimension $d \times d$. Importantly, the expected update direction of the ZO method is equivalent to performing SGD on smoothed loss $\mathcal{L}_x^\mu(\theta)$ (Lemma L.1). This equivalence establishes the theoretical foundation for analyzing the convergence rate of SZO for SNN. Now, we introduce the assumption for analyzing the convergence rate.

Assumption 5.2 (Basic Assumptions). We impose the following fundamental assumptions on the smoothed loss function $\mathcal{L}^\mu$ and the projection matrix P :

1. The smoothed loss function $\mathcal{L}_x^\mu(\theta)$ is L -smooth with respect to θ and $L \geq 1$ without loss of generality. We also assume the gradient norm is uniformly bounded, i.e., $\|\nabla \mathcal{L}_x^\mu(\theta)\| \leq G$ for all $\theta \in \mathbb{R}^d$. This also implies $\|\nabla \mathcal{L}^\mu(\theta)\| \leq G$ for all $\theta \in \mathbb{R}^d$.
2. For the projection matrix P , throughout the training process of T steps, there exists $\varepsilon > 0$ such that $\|PP^\top \nabla \mathcal{L}_x^\mu(\theta_t) - \nabla \mathcal{L}_x^\mu(\theta_t)\| \leq \varepsilon$ for all training step t and $x \sim \mathcal{D}$. Consequently, $\|PP^\top \nabla \mathcal{L}_B^\mu(\theta_t) - \nabla \mathcal{L}_B^\mu(\theta_t)\| \leq \varepsilon$ and $\|PP^\top \nabla \mathcal{L}^\mu(\theta_t) - \nabla \mathcal{L}^\mu(\theta_t)\| \leq \varepsilon$ also holds.
3. The variance due to sampling x is bounded by σ^2 for any θ , i.e., $\mathbb{E}_x[\|\nabla \mathcal{L}_x^\mu(\theta) - \nabla \mathcal{L}^\mu(\theta)\|^2] \leq \sigma^2$, which further implies $\mathbb{E}_B[\|\nabla \mathcal{L}_B^\mu(\theta) - \nabla \mathcal{L}^\mu(\theta)\|^2] \leq \sigma^2/B$.

The first assumption regarding the loss function is a standard requirement for convergence rate analysis in the context of non-convex loss functions and momentum-based update rules. The second assumption ensures that the gradient can be projected into the subspace and subsequently recovered with only a small discrepancy. This aligns with our empirical results, which indicate that the optimization trajectory of neural networks in classification tasks predominantly lies

within a subspace, and the projection matrix captures the basis of this subspace. The third assumption is a common condition in the analysis of mini-batch SGD algorithms.

To establish the convergence rate of the algorithm, we first present the following lemma.

Lemma 5.3 (Estimation error of the SZO gradient estimator). Suppose the smoothed loss function $\mathcal{L}^\mu$ and the projection matrix P satisfy Assumption 5.1 and 5.2, and let $\mathcal{B}$ be a mini-batch data of size B . Then, under the gradient estimator (5) and parameter update rule given (6), for any parameter θ_t , the following bound holds:

$$\mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_B(\theta_t) - \nabla \mathcal{L}_B^\mu(\theta_t) \right\|^2 \right] \leq \frac{9M^2k}{\mu^2} + 2\varepsilon^2.$$

Lemma 5.3 demonstrates that the discrepancy in the SZO gradient estimator and the corresponding gradient of the smoothed loss. Leveraging this lemma, we subsequently derive the overall convergence rate.

Theorem 5.4 (Convergence rate of SZO). Suppose the loss function $\mathcal{L}$ and the projection matrix P satisfy Assumption 5.1 and 5.2. Consider minimizing $\mathcal{L}$ using mini-batch gradients over T training steps, with momentum parameter β , learning rate $\eta = \frac{1-\beta}{L\sqrt{T}}$, the perturbation scale $\mu \geq 6M\sqrt{\frac{L\sqrt{T}}{1-\beta}}$, and the projection error $\varepsilon \leq \sqrt{\frac{1-\beta}{8L\sqrt{T}}}$. Under the update rule (6) and choose $T > \left(\frac{k+5}{2}\right)^2$, the convergence rate of the algorithm satisfies

$$\begin{aligned} & \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} [\|\nabla \mathcal{L}^\mu(\theta_t)\|^2] \\ & \leq \mathcal{O} \left(\frac{k(1-\beta)}{\sqrt{T}} + \left(\frac{1}{1-\beta} \right)^3 \frac{\sigma^2}{B\sqrt{T}} \right). \end{aligned}$$

Theorem 5.4 demonstrates that subspace-based ZO achieves a convergence rate of $\mathcal{O}(k/\sqrt{T})$, which offers a substantial improvement over the full-space ZO optimization rate of $\mathcal{O}(d/\sqrt{T})$, as $k \ll d$.

Crucially, this theorem comes with a key insight: the convergence guarantee requires a relatively large perturbation scale μ , contrasting sharply with standard ANN convergence analysis where small μ is typically required. This reveals a fundamental methodological difference between SNN and ANN in ZO optimization.

6. Experiments

We evaluate the proposed method on three tasks: model training from scratch, continual training, and unsupervised adaptation. We consider four SNN models, Spike-ResNet20 (0.27M), Spike-ResNet18 (11.2M), Spike-ResNet50 (25.56M) (Hu et al., 2025), and Spike-VGG11

Table 1. Accuracy (%) comparison on training from scratch.

Dataset	Model	Params	BP	BP-Subspace	SZO-CGE	SZO-RGE($q=1$)	SZO-RGE($q=20$)
CIFAR10	Spike-ResNet20	0.27M	86.74 $\pm$ 0.12	86.42 $\pm$ 0.32	86.21 $\pm$ 0.17	81.29 $\pm$ 0.35	83.71 $\pm$ 0.34
CIFAR100	Spike-ResNet18	11.22M	74.51 $\pm$ 0.16	73.32 $\pm$ 0.22	73.35 $\pm$ 0.30	73.77 $\pm$ 0.26	73.79 $\pm$ 0.22
DVS-Gesture	Spike-VGG11	9.50M	97.57 $\pm$ 0.35	96.65 $\pm$ 1.06	96.65 $\pm$ 0.40	91.78 $\pm$ 1.64	95.72 $\pm$ 0.40
ImageNet1K	Spike-ResNet50	25.56M	63.81 $\pm$ 0.78	61.71 $\pm$ 0.30	61.71 $\pm$ 0.18	61.82 $\pm$ 0.10	61.92 $\pm$ 0.08

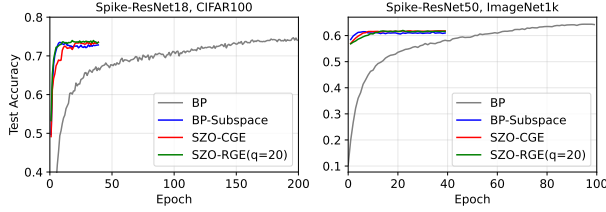


Figure 3. Training dynamics of different methods.

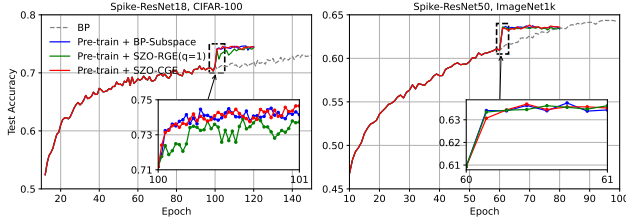


Figure 4. Test accuracy of the compared methods on continual training task.

(9.5M), which are trained and tested on CIFAR10, CIFAR100, ImageNet1K and DVS-Gesture datasets, respectively. We take 4 time-steps for static datasets and 20 time-steps for DVS-Gesture. In SZO, we use a perturbation scale $\mu = 1$, a momentum coefficient $\beta = 0.9$, without weight decay. The learning rate is decayed using a cosine method, with an initial value of 1. We compare SZO with full-space BP and subspace-based BP, which are trained by BPTT with surrogate gradient. Moreover, subspace-based BP and SZO use the same subspace. For subspace-based BP, the parameters are updated using the BP gradient projected into the subspace. We employ the TET loss function (Deng et al., 2022) for all supervised training ($\lambda = 0.05$, $\phi = 1$), and the entropy loss (Wang et al., 2021) for unsupervised model adaptation, maintaining identical running statistics of all BN layers for all perturbations in one time step. More training details gives in Appendix J. We use SpikingJelly (Fang et al., 2023a) for SNN training and all experiments are conducted on eight NVIDIA V100 GPUs, each with 32GB of memory.

6.1. Training from Scratch in Subspace

For CIFAR10 and CIFAR100, the model sampling number is $N = 150$, and the subspace dimension is $k = 60$. For DVS-Gesture, the model sampling number is $N = 80$, and the subspace dimension $k = 20$. The models are sampled ev-

ery epoch from the full-space BP training. For ImageNet1K, we sample $N = 150$ models from epoch 0 to 60 uniformly and extract a subspace dimension of $k = 60$.

SZO and subspace-based BP are trained for 40 epochs on all datasets, while BP is trained for 100 epochs on ImageNet1K and 200 epochs on other datasets to converge. Table 1 and Figure 3 show that SZO can achieve comparable performance with that of BP and subspace-based BP (BP-Subspace). In particular, subspace-based training converges significantly faster than full-space BP. For instance, on ImageNet-1K, training in the subspace for fewer than 10 epochs can achieve the similar accuracy as 60 epochs of full-space BP. This rapid learning capability is particularly desirable for on-chip learning scenarios.

CGE typically outperforms RGE in high-dimensional settings such as Chen et al. (2024). However, SZO operates in a very low-dimensional subspace (tens), which largely eliminates the difference between SZO-CGE and SZO-RGE.

6.2. Model Continual Training

To verify that the extracted subspace enhances ZO continual training, we first pre-train the Spike-ResNet18 model with full-space BP for 100 epochs, sampling from epochs 80 to 100 to create a 20-dimensional subspace for CIFAR100 using PCA. For ImageNet1K, we pre-train Spike-ResNet50 for 60 epochs and extract a 20-dimensional subspace by sampling from epoch 50 to 60. Continual training is then performed using subspace-based methods for 20 epochs based on the pre-trained model. Figure 4 compares the continual training performance of SZO with BP.

It can be seen that, after switching to subspace-based training, SZO exhibits a sharp increase in accuracy within a single epoch on both CIFAR100 and ImageNet1K. On CIFAR100, SZO achieves an accuracy exceeding 74% in just one epoch, which closely approaches the converged accuracy of full-space BP after approximately 50 epochs. This demonstrates the significant acceleration enabled by subspace-based learning. The improved convergence rate is theoretically justified in Section 5.

Finally, on CIFAR100 SZO-CGE continual training achieves an accuracy of 74.59%, which is even higher than that of full-space BP (74.42%). On ImageNet1K, SZO-CGE achieves an accuracy of 63.73%, approaching the converged accuracy of full-space BP (64.36%). SZO-RGE achieves

Table 2. Accuracy (%) comparison on unsupervised model adaptation on CIFAR10C, CIFAR100C and ImageNetC (all with severity level 5).

Setting	Dataset	Model	No adapt.	BN	Tent (BP)	BP-Subspace	SZO-CGE	SZO-RGE ($q=10$)
<i>Reset</i>	CIFAR10C	Spike-ResNet20	45.01	65.53	66.14	67.69	67.09	57.09
	CIFAR100C	Spike-ResNet18	29.81	51.72	52.54	75.02	72.47	63.03
	ImageNetC	Spike-ResNet50	9.36	21.20	22.82	31.76	32.94	32.34
<i>Continual</i>	CIFAR10C	Spike-ResNet20	45.01	65.53	64.58	72.73	76.10	31.09
	CIFAR100C	Spike-ResNet18	29.81	51.72	53.84	78.15	79.69	75.86
	ImageNetC	Spike-ResNet50	9.36	21.20	3.89	32.15	33.37	33.16

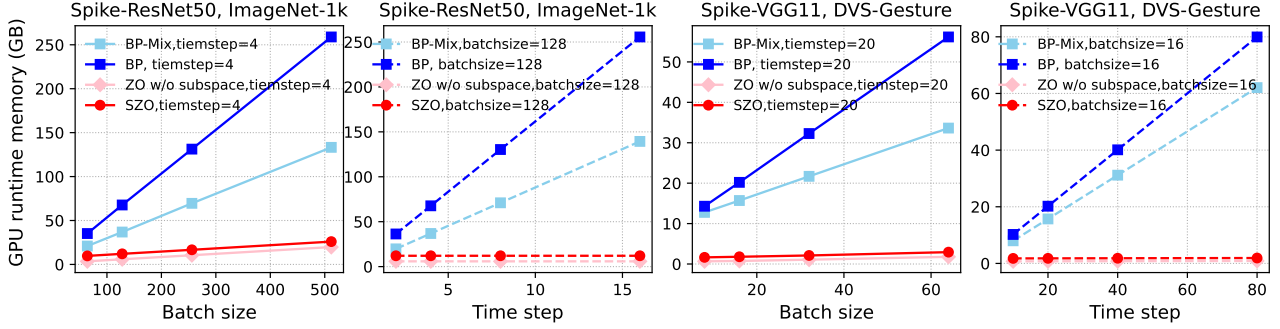


Figure 5. GPU runtime memory under varying batch sizes and time steps.

comparable maximum accuracy to SZO-CGE, but requires less training time, as detailed in Table 3.

6.3. Unsupervised Model Adaptation

We further consider unsupervised model adaptation, especially test-time adaptation (TTA) (Schneider et al., 2020; Wang et al., 2021), which adapts a source model to a target domain using only test data in an unsupervised manner. We evaluate pretrained Spike-ResNet20, Spike-ResNet18 and Spike-ResNet50 models on CIFAR10C, CIFAR100C and ImageNetC, respectively. CIFAR100C and ImageNetC each contain 15 corruption types. Models are adapted over a single cycle of the 15 corruptions. In this cross-domain setting, while using the subspace extracted from source pre-training can also significantly improve cross-domain adaptation performance, relying on it alone remains insufficient to achieve optimal results comparable to state-of-the-art methods. Thus, we construct a subspace specialized for corruption adaptation. Rather than building one for each corruption, we find that using a representative corruption suffices for generalization. Specifically, for ImageNetC, we select Gaussian (severity 5), and sample 20 BP-trained models in 20 supervised training epochs. Then, these models are combined with 20 source models to extract the subspace with 20 dimensions using PCA. For CIFAR10C and CIFAR100C, we only sample 20 models trained on Gaussian (severity 1) in 10 supervised training epochs, combining with 20 source models and reduce dimension to 20 using PCA. We evaluate under *Reset* and *Continual* settings for each method. The *Reset* setting means the model is reset to its ini-

tial parameters after adapting to one corruption type, while the *Continual* setting means the model continuously adapts to the 15 corruption types without resetting.

Table 2 compares SZO with two representative TTA methods, BN (Schneider et al., 2020) and Tent (Wang et al., 2021). More results are provided in Appendix E. SZO achieves significant better performance than Tent trained with full-space BP. The better performance of SZO is attributed to the stability provided by training in low-dimensional subspace, which mitigates the instability of using unsupervised entropy loss as entropy minimization may be misled by samples with high confidence (i.e., low entropy) but incorrect predictions (Duan et al., 2025). This advantage of subspace training is more obvious under the *Continual* setting. These results demonstrate that SZO is effective when the subspace is extracted from source tasks that share similarity with the downstream tasks.

6.4. Computational Efficiency

GPU runtime memory. GPU memory of BP, BP with mixed-precision (BP-Mix), SZO and ZO without subspace, recorded via `torch.cuda.max_memory_allocated()` are shown in Figure 5. As BP and BP-Mix must store all inter-layer activations, its memory usage grows steeply with both batch-size and time-steps. In contrast, SZO does not store these activations, thus its memory usage increases only marginally as batch-size increases and remains almost invariant to time-step. Although SZO need to store subspace projection matrix, this overhead is significantly smaller than the activation storage required by BP and BP-mix.

Table 3. Comparison of training time in the continual training and training from scratch task. #FP and #BP denote the number of forward and backward passes. For CIFAR100, all methods run on a single V100 GPU. For ImageNet1K, BP and BP-subspace use 8 V100s, while SZO-CGE/RGE use 2 V100s.

Dataset / Model	Method	Continual training					Training from scratch				
		Converged epoch	Converged time (min)	#FP	#BP	Max Acc(%)	Converged epoch	Converged time (min)	#FP	#BP	Max Acc(%)
CIFAR100 / Spike-ResNet18	BP	97	58.2	30107	30107	74.42	197	142.8	77027	77027	74.42
	BP-Subspace	4	3.4	1564	1564	74.64	8	9.4	3128	3128	73.37
	SZO-CGE	4	69.6	64124	0	74.59	38	1335.2	1797818	0	73.42
	SZO-RGE($q=1$)	6	8.7	7038	0	74.42	39	60.7	45747	0	73.91
ImageNet1K / Spike-ResNet50	BP	35	1248.5	41316	41316	64.36	95	3939.6	118940	118940	64.36
	BP-Subspace	1	41.6	1252	1252	63.74	7	324.8	8764	8764	61.49
	SZO-CGE	3	1104.3	78876	0	63.73	18	32011.2	2726856	0	61.84
	SZO-RGE($q=1$)	1	56.5	3756	0	63.62	28	1484.6	105168	0	61.78

Table 4. Accuracy (%) of SZO-CGE on training from scratch task with different perturbation scales.

Dataset	Network	$\mu = 0.6$	$\mu = 1.0$	$\mu = 1.4$	$\mu = 1.8$	$\mu = 2.2$
CIFAR10	Spike-ResNet20	85.91 $\pm$ 0.31	86.21 $\pm$ 0.17	86.01 $\pm$ 0.41	86.03 $\pm$ 0.29	85.92 $\pm$ 0.12
CIFAR100	Spike-ResNet18	73.20 $\pm$ 0.33	73.35 $\pm$ 0.30	73.09 $\pm$ 0.41	73.07 $\pm$ 0.24	72.98 $\pm$ 0.47

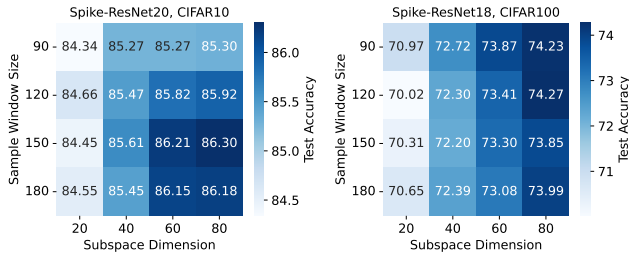


Figure 6. Accuracy of SZO-CGE with different sample window size and subspace dimension.

Training time. Table 3 compares the training time to reach maximum accuracy in the continual training and training from scratch tasks. The training time for continual training is measured starting from the beginning of the continual training stage. Although SZO involves more forward computations per iteration, it converges faster and thus reaches maximum accuracy in fewer epochs. Moreover, we found that SZO-RGE further reduces the number of forward passes while achieving comparable accuracy in most cases.

6.5. Ablation study

Perturbation scale. Table 4 shows the accuracy of SZO-CGE in the training from scratch experiment for different perturbation scales. It performs well on a wide range of perturbation scales. The result validates our theoretical finding in Theorem 5.4 that μ need to be relatively large for effective SNN optimization, which directly contrasts with ANN requirements where small μ is standard.

Model sampling window size and subspace dimension. In constructing the subspace, the models are typically sampled once per epoch starting from the initial weight. A smaller window size N means that fewer source-pretraining epochs are needed, while a lower subspace dimension k results in

less memory overhead. Figure 6 shows that SZO-CGE using a larger subspace dimension can achieve higher accuracy in training from scratch task.

7. Discussion on Efficient Neuromorphic On-Chip Learning Enabled by SZO

SZO is motivated to overcome the barriers that make BP unsuitable for hardware deployment: the need for dedicated feedback circuits, the update-locking problem, high memory overhead, and inefficient weight transport, as detailed in Section 1. In contrast, SZO relies only on forward computation and offers strong suitability for practical on-chip learning: (1) *No dedicated feedback circuits are required.* (2) Learning in a low-dimensional subspace enables significantly *faster convergence* compared to full-space BP, which is particularly desirable for real-time on-chip learning. (3) Although SZO requires storing a subspace matrix of size $d \times k$, it eliminates the need to store activations, thus has *far lower memory overhead than BP* (Figure 5). (4) Unlike vanilla ZO methods that require repeatedly sampling high-dimensional random vectors of size d , SZO only samples within a subspace of dimension k (typically tens, e.g., $k = 20, 60$ vs. $d = 11.22\text{M}$ and 25.56M for Spike-ResNet18 and Spike-ResNet50). This dimensionality reduction not only accelerates optimization but also *removes the hardware bottleneck of generating and manipulating extremely high-dimensional random perturbations*, which makes ZO learning substantially more hardware-efficient and practically deployable on neuromorphic chips.

8. Conclusion

In this work, we revealed the underlying cause of the ineffectiveness ZO optimization on training SNNs, and proposed a

subspace-based ZO method for SNN training. Further, we provided theoretical analysis to show that subspace-based ZO training can improve the convergence rate compared with the full-space ZO training. Experiments on model training from scratch, continual training, and unsupervised adaptation demonstrated that SZO not only closely approaches FO training performance but also offering much faster learning speed, which is particularly desirable for on-chip learning. SZO is the first forward-passes-only SNN training method that can scale to large-scale datasets such as ImageNet1K.

Limitations: In the proposed SZO method, the subspace is constructed in a hand-crafted manner from offline BP pretraining. While this design is effective in our experiments, its effectiveness depends on the transferability of the resulting subspace to downstream on-chip learning tasks. Exploring data-driven or meta-learning based approaches can be expected to construct more transferable subspace. Moreover, our study mainly focuses on small-to-medium-scale models and classification tasks. Extending the proposed method to larger models and tasks such as sequential language modeling is an important direction for future work.

Acknowledgments

This work was supported in part by the Brain Science and Brain-like Intelligence Technology–National Science and Technology Major Project under Grant 2022ZD0208700 and the National Natural Science Foundation of China (NSFC) under Grant 62271314.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, more specifically neuromorphic on-chip learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Akrout, M., Wilson, C., Humphreys, P. C., Lillicrap, T. P., and Tweed, D. Deep learning without weight transport. In *Advances in Neural Information Processing Systems* 32, 2019.
- Ali, F., Otchy, T. M., Pehlevan, C., Fantana, A. L., Burak, Y., and Ölveczky, B. P. The basal ganglia is necessary for learning spectral, but not temporal, features of birdsong. *Neuron*, 80(2):494–506, 2013.
- Aronov, D., Andalman, A. S., and Fee, M. S. A specialized forebrain circuit for vocal babbling in the juvenile songbird. *Science*, 320(5876):630–634, 2008.
- Bacho, F. and Chu, D. Low-variance forward gradients using direct feedback alignment and momentum. *Neural Networks*, 169:572–583, 2024.
- Baek, E., Lee, H., Kim, Y., and Kim, J. Flexlearn: fast and highly efficient brain simulations using flexible on-chip learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 304–318, 2019.
- Baydin, A. G., Pearlmutter, B. A., Syme, D., Wood, F., and Torr, P. Gradients without backpropagation. *arXiv preprint arXiv:2202.08587*, 2022.
- Bellec, G., Scherr, F., Subramoney, A., Hajek, E., Salaj, D., Legenstein, R., and Maass, W. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11(1):3625, 2020.
- Bi, G.-q. and Poo, M.-m. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of neuroscience*, 18(24):10464–10472, 1998.
- Billaudelle, S., Stradmann, Y., Schreiber, K., Cramer, B., Baumbach, A., Dold, D., Göltz, J., Kungl, A. F., Wunderlich, T. C., Hartel, A., Müller, E., Breitwieser, O., Mauch, C., Kleider, M., Grübl, A., Stöckel, D., Pehle, C., Heimbrecht, A., Spilger, P., Kiene, G., Karasenko, V., Senn, W., Petrovici, M. A., Schemmel, J., and Meier, K. Versatile emulation of spiking neural networks on an accelerated neuromorphic substrate. In *IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2020.
- Bohnstingl, T., Wozniak, S., Pantazi, A., and Eleftheriou, E. Online spatio-temporal learning in deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8894–8908, 2022.
- Bouvier, G., Aljadeff, J., Clopath, C., Bimbard, C., Ranft, J., Blot, A., Nadal, J.-P., Brunel, N., Hakim, V., and Barbour, B. Cerebellar learning using perturbations. *eLife*, 7: e31599, 2018.
- Bu, T., Fang, W., Ding, J., DAI, P., Yu, Z., and Huang, T. Optimal ann-snn conversion for high-accuracy and ultra-low-latency spiking neural networks. In *International Conference on Learning Representations*, 2022.
- Burkitt, A. N. A review of the integrate-and-fire neuron model: I. homogeneous synaptic input. *Biological Cybernetics*, 95:1–19, 2006.
- Chen, A., Zhang, Y., Jia, J., Diffenderfer, J., Parasyris, K., Liu, J., Zhang, Y., Zhang, Z., Kailkhura, B., and Liu, S. Deepzero: Scaling up zeroth-order optimization for deep model training. In *International Conference on Learning Representations (ICLR)*, 2024.

- Chen, F., Tian, Q., Xie, L., Zhou, Y., Wu, Z., Wu, L., Ying, R., Wen, F., and Liu, P. Epoc: A 28-nm 5.3 pj/sop event-driven parallel neuromorphic hardware with neuromodulation-based online learning. *IEEE Transactions on Biomedical Circuits and Systems*, 19(3):629–644, 2025a.
- Chen, Y., Zhang, Y., Cao, L., Yuan, K., and Wen, Z. Enhancing zeroth-order fine-tuning for language models with low-rank structures. In *International Conference on Learning Representations*, volume 2025, pp. 62581–62607, 2025b.
- Christensen, D. V., Dittmann, R., Linares-Barranco, B., Sebastian, A., Le Gallo, M., Redaelli, A., Slesazeck, S., Mikolajick, T., Spiga, S., Menzel, S., Valov, I., Milano, G., Ricciardi, C., Liang, S.-J., Miao, F., Lanza, M., Quill, T. J., Keene, S. T., Salleo, A., Grollier, J., Marković, D., Mizrahi, A., Yao, P., Yang, J. J., Indiveri, G., Strachan, J. P., Datta, S., Vianello, E., Valentian, A., Feldmann, J., Li, X., Pernice, W. H. P., Bhaskaran, H., Furber, S., Neftci, E., Scherr, F., Maass, W., Ramaswamy, S., Tapson, J., Panda, P., Kim, Y., Tanaka, G., Thorpe, S., Bartolozzi, C., Cleland, T. A., Posch, C., Liu, S.-C., Panuccio, G., Mahmud, M., Mazumder, A. N., Hosseini, M., Mohsenin, T., Donati, E., Tolu, S., Galeazzi, R., Christensen, M. E., Holm, S., Ielmini, D., and Pryds, N. 2022 roadmap on neuromorphic computing and engineering. *Neuromorphic Computing and Engineering*, 2(2):022501, 2022.
- Davies, M., Srinivasa, N., Lin, T.-H., China, G. N., Cao, Y., Choday, S. H., Dimou, G. D., Joshi, P., Imam, N., Jain, S., Liao, Y., Lin, C.-K., Lines, A., Liu, R., Mathaikutty, D., McCoy, S., Paul, A., Tse, J., Venkataramanan, G., Weng, Y.-H., Wild, A., Yang, Y., and Wang, H. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.
- Deng, S., Li, Y., Zhang, S., and Gu, S. Temporal efficient training of spiking neural network via gradient re-weighting. In *International Conference on Learning Representations*, 2022.
- Duan, D., Xu, R., Liu, P., and Wen, F. Lifelong test-time adaptation via online learning in tracked low-dimensional subspace. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Duchi, J. C., Jordan, M. I., Wainwright, M. J., and Wibisono, A. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015.
- Fang, W., Yu, Z., Chen, Y., Huang, T., Masquelier, T., and Tian, Y. Deep residual learning in spiking neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pp. 21056–21069, 2021.
- Fang, W., Chen, Y., Ding, J., Yu, Z., Masquelier, T., Chen, D., Huang, L., Zhou, H., Li, G., and Tian, Y. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023a. doi: 10.1126/sciadv.adi1480. URL <https://www.science.org/doi/abs/10.1126/sciadv.adi1480>.
- Fang, W., Yu, Z., Zhou, Z., Chen, D., Chen, Y., Ma, Z., Masquelier, T., and Tian, Y. Parallel spiking neurons with high efficiency and ability to learn long-term dependencies. In *Advances in Neural Information Processing Systems*, 2023b.
- Fiete, I. R., Fee, M. S., and Seung, H. S. Model of bird-song learning based on gradient estimation by dynamic perturbation of neural conductances. *Journal of Neurophysiology*, 98(4):2038–2057, 2007.
- Flower, B. and Jabri, M. Summed weight neuron perturbation: an $\mathcal{O}(n)$ improvement over weight perturbation. In *Advances in Neural Information Processing Systems 5 (NIPS 1992)*, pp. 212–219, 1992.
- Fournier, L., Rivaud, S., Belilovsky, E., Eickenberg, M., and Oyalon, E. Can forward gradient match backpropagation? In *International Conference on Machine Learning*, pp. 10249–10264, 2023.
- Frenkel, C. and Indiveri, G. Reckon: A 28nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 190–192. IEEE, 2022.
- Frenkel, C., Lefebvre, M., Legat, J.-D., and Bol, D. A 0.086-mm² 12.7-pj/sop 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm cmos. *IEEE Transactions on Biomedical Circuits and Systems*, 13(1):145–158, 2018.
- Frenkel, C., Legat, J.-D., and Bol, D. Morphic: a 65-nm 738k-synapse/mm² quad-core binary-weight digital neuromorphic processor with stochastic spike-driven online learning. *IEEE Transactions on Biomedical Circuits and Systems*, 13(5):999–1010, 2019.
- Frenkel, C., Lefebvre, M., and Bol, D. Learning without feedback: fixed random learning signals allow for feed-forward training of deep neural networks. *Frontiers in Neuroscience*, 15:629892, 2021.
- Furber, S. B., Galluppi, F., Temple, S., and Plana, L. A. The spinnaker project. *Proceedings of the IEEE*, 102(5): 652–665, 2014.

- Gressmann, F., Eaton-Rosen, Z., and Luschi, C. Improving neural network training in low dimensional random bases. *Advances in Neural Information Processing Systems*, 33: 12140–12150, 2020.
- Gu, Q. Neuromodulatory transmitter systems in the cortex and their role in cortical plasticity. *Neuroscience*, 111(4): 815–835, 2002.
- Guo, W., Fouda, M. E., Eltawil, A. M., and Salama, K. N. Efficient neuromorphic hardware through spiking temporal online local learning. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 30(11):1642–1653, 2022.
- Guo, W., Long, J., Zeng, Y., Liu, Z., Yang, X., Ran, Y., Gardner, J. R., Bastani, O., Sa, C. D., Yu, X., Chen, B., and Xu, Z. Zeroth-order fine-tuning of LLMs with transferable static sparsity. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Gur-Ari, G., Roberts, D. A., and Dyer, E. Gradient descent happens in a tiny subspace. *arXiv preprint arXiv:1812.04754*, 2018.
- Han, B., Srinivasan, G., and Roy, K. RMP-SNN: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13558–13567, 2020.
- Heidarpur, M., Ahmadi, A., Ahmadi, M., and Rahimi Azghadi, M. Cordic-snn: On-fpga stdp learning with izhikevich neurons. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 66(7): 2651–2661, 2019.
- Hiratani, N., Mehta, Y., Lillicrap, T. P., and Latham, P. E. On the stability and scalability of node perturbation learning. In *Advances in Neural Information Processing Systems* 35, 2022.
- Hu, E. J., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Hu, Y., Tang, H., and Pan, G. Spiking deep residual networks. *IEEE Transactions on Neural Networks and Learning Systems*, 34(8):5200–5205, 2021.
- Hu, Y., Zheng, Q., Jiang, X., and Pan, G. Fast-snn: Fast spiking neural network by converting quantized ann. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(12):14546–14562, 2023.
- Hu, Y., Deng, L., Wu, Y., Yao, M., and Li, G. Advancing spiking neural networks toward deep residual learning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(2):2353–2367, 2025.
- Jin, F., Liu, Y., and Tan, Y. Derivative-free optimization for low-rank adaptation in large language models. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:4607–4616, 2024.
- Kaiser, J., Mostafa, H., and Neftci, E. Synaptic plasticity dynamics for deep continuous local learning (decolle). *Frontiers in Neuroscience*, 14:424, 2020.
- Kohonen, T. Self-organized formation of topologically correct feature maps. *Biological cybernetics*, 43(1):59–69, 1982.
- Krotov, D. and Hopfield, J. J. Unsupervised learning by competing hidden units. *Proceedings of the National Academy of Sciences*, 116(16):7723–7731, 2019.
- Larsen, B. W., Fort, S., Becker, N., and Ganguli, S. How many degrees of freedom do we need to train deep networks: a loss landscape perspective. In *International Conference on Learning Representations*, 2022.
- Launay, J., Poli, I., Boniface, F., and Krzakala, F. Direct feedback alignment scales to modern deep learning tasks and architectures. In *Advances in Neural Information Processing Systems* 33, 2020.
- Lee, C., Sarwar, S. S., Panda, P., Srinivasan, G., and Roy, K. Enabling spike-based backpropagation for training deep neural network architectures. *Frontiers in Neuroscience*, 14:119, 2020a.
- Lee, D.-H., Zhang, S., Fischer, A., and Bengio, Y. Difference target propagation. In *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, pp. 498–515. Springer, 2015.
- Lee, J. H., Delbruck, T., and Pfeiffer, M. Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, 10:508, 2016.
- Lee, J.-J., Zhang, R., Zhang, W., Liu, Y., and Li, P. Spike-train level direct feedback alignment: sidestepping backpropagation for on-chip training of spiking neural nets. *Frontiers in Neuroscience*, 14:143, 2020b.
- Li, C., Farkhoor, H., Liu, R., and Yosinski, J. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations*, 2018.

- Li, T., Tan, L., Huang, Z., Tao, Q., Liu, Y., and Huang, X. Low dimensional trajectory hypothesis is true: Dnns can be trained in tiny subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3):3411–3420, 2022.
- Li, Y., Guo, Y., Zhang, S., Deng, S., Hai, Y., and Gu, S. Differentiable spike: Rethinking gradient-descent for training spiking neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pp. 23426–23439, 2021.
- Liang, L., Qu, Z., Chen, Z., Tu, F., Wu, Y., Deng, L., Li, G., Li, P., and Xie, Y. H2learn: High-efficiency learning accelerator for high-accuracy spiking neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(11):4782–4796, 2021.
- Liao, Q., Leibo, J. Z., and Poggio, T. How important is weight symmetry in backpropagation? In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1837–1844, 2016.
- Lillicrap, T. P., Cownden, D., Tweed, D. B., and Akerman, C. J. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7(1):13276, 2016.
- Lillicrap, T. P., Santoro, A., Marris, L., Akerman, C., and Hinton, G. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.
- London, M., Roth, A., Beeren, L., Häusser, M., and Latham, P. E. Sensitivity to perturbations in vivo implies high noise and suggests rate coding in cortex. *Nature*, 466(7302):123–127, 2010.
- Maass, W. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9):1659–1671, 1997.
- Magee, J. C. and Grienberger, C. Synaptic plasticity forms and functions. *Annual review of neuroscience*, 43(1):95–117, 2020.
- Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems*, 2023.
- Markram, H., Lübke, J., Frotscher, M., and Sakmann, B. Regulation of synaptic efficacy by coincidence of post-synaptic apss and epsps. *Science*, 275(5297):213–215, 1997.
- Martin, E., Ernoul, M., Laydevant, J., Li, S., Querlioz, D., Petrisor, T., and Grollier, J. Eqspike: spike-driven equilibrium propagation for neuromorphic implementations. *Science*, 24(3), 2021.
- Mazzoni, P., Andersen, R. A., and Jordan, M. I. A more biologically plausible learning rule for neural networks. *Proceedings of the National Academy of Sciences*, 88(10):4433–4437, 1991.
- Meng, Q., Xiao, M., Yan, S., Wang, Y., Lin, Z., and Luo, Z.-Q. Towards memory- and time-efficient backpropagation for training spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6166–6176, 2023.
- Miconi, T. Biologically plausible learning in recurrent neural networks reproduces neural dynamics observed during cognitive tasks. *eLife*, 6:e20899, 2017.
- Mukhoty, B., Bojkovic, V., de Vazelhes, W., Zhao, X., De Masi, G., Xiong, H., and Gu, B. Direct training of snn using local zeroth order method. *Advances in Neural Information Processing Systems*, 36:18994–19014, 2023.
- Neftci, E. O., Mostafa, H., and Zenke, F. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.
- Nøkland, A. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems* 29, pp. 1037–1045, 2016.
- Oja, E. Simplified neuron model as a principal component analyzer. *Journal of mathematical biology*, 15(3):267–273, 1982.
- Park, J., Lee, J., and Jeon, D. A 65-nm neuromorphic image classification processor with energy-efficient training through direct spike-only feedback. *IEEE Journal of Solid-State Circuits*, 55(1):108–119, 2019.
- Pérez-Carrasco, J. A., Zhao, B., Serrano, C., Acha, B., Serrano-Gotarredona, T., Chen, S., and Linares-Barranco, B. Mapping from frame-driven to frame-free event-driven vision systems by low-rate rate coding and coincidence processing—application to feedforward convnets. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(11):2706–2719, 2013.
- Qiao, N., Mostafa, H., Corradi, F., Osswald, M., Stefanini, F., Sumislawska, D., and Indiveri, G. A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128k synapses. *Frontiers in Neuroscience*, 9:141, 2015.
- Radhakrishnan, S. S., Sebastian, A., Oberoi, A., Das, S., and Das, S. A biomimetic neural encoder for spiking neural network. *Nature Communications*, 12(1):2143, 2021.
- Rathi, N. and Roy, K. DIET-SNN: A low-latency spiking neural network with direct input encoding and leakage

- and threshold optimization. *IEEE Transactions on Neural Networks and Learning Systems*, 34(6):3174–3182, 2021.
- Ren, M., Kornblith, S., Liao, R., and Hinton, G. Scaling forward gradient with local losses. In *International Conference on Learning Representations*, 2023.
- Rueckauer, B., Lungu, I., Hu, Y., Pfeiffer, M., and Liu, S. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, 11:682, 2017.
- Scellier, B. and Bengio, Y. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in computational neuroscience*, 11:24, 2017.
- Schemmel, J., Brüderle, D., Grübl, A., Hock, M., Meier, K., and Millner, S. A wafer-scale neuromorphic hardware system for large-scale neural modeling. In *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1947–1950, 2010.
- Schneider, S., Rusak, E., Eck, L., Bringmann, O., Brendel, W., and Bethge, M. Improving robustness against common corruptions by covariate shift adaptation. *Advances in Neural Information Processing Systems*, 33: 11539–11551, 2020.
- Seung, H. S. Learning in spiking neural networks by reinforcement of stochastic synaptic transmission. *Neuron*, 40(6):1063–1073, 2003.
- Song, S. and Abbott, L. F. Cortical development and remapping through spike timing-dependent plasticity. *Neuron*, 32(2):339–350, 2001.
- Von der Malsburg, C. Self-organization of orientation sensitive cells in the striate cortex. *Kybernetik*, 14(2):85–100, 1973.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- Wu, J., Xu, C., Han, X., Zhou, D., Zhang, M., Li, H., and Tan, K. C. Progressive tandem learning for pattern recognition with deep spiking neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7824–7840, 2022.
- Wu, Y., Deng, L., Li, G., Zhu, J., and Shi, L. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12: 331, 2018.
- Xiao, M., Meng, Q., Zhang, Z., He, D., and Lin, Z. On-line training through time for spiking neural networks. In *Advances in Neural Information Processing Systems*, volume 35, pp. 20717–20730, 2022.
- Xiao, M., Meng, Q., Zhang, Z., He, D., and Lin, Z. Online pseudo-zeroth-order training of neuromorphic spiking neural networks. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Xiao, W., Chen, H., Liao, Q., and Poggio, T. Biologically-plausible learning algorithms can scale to large datasets. In *International Conference on Learning Representations (ICLR)*, 2019.
- Yao, M., Hu, J., Hu, T., Xu, Y., Zhou, Z., Tian, Y., Xu, B., and Li, G. Spike-driven transformer v2: Meta spiking neural network architecture inspiring the design of next-generation neuromorphic chips. In *International Conference on Learning Representations*, 2024.
- Yin, B., Corradi, F., and Bohté, S. M. Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks. *Nature Machine Intelligence*, 3(10): 905–913, 2021.
- Yin, B., Corradi, F., and Bohté, S. M. Accurate online training of dynamical spiking neural networks through forward propagation through time. *Nature Machine Intelligence*, 5(5):518–527, 2023.
- Yu, Z., Zhou, P., Wang, S., Li, J., Tian, M., and Huang, H. Zeroth-order fine-tuning of llms in random subspaces. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4475–4485, 2025.
- Zenke, F. and Ganguli, S. Superspike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6):1514–1541, 2018.
- Zhang, H. and Zhang, Y. Memory-efficient reversible spiking neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 16759–16767, 2024.
- Zhang, J., Huo, D., Zhang, J., Qian, C., Liu, Q., Pan, L., Wang, Z., Qiao, N., Tang, K.-T., and Chen, H. Anp-i: A 28-nm 1.5-pj/sop asynchronous spiking neural network processor enabling sub-0.1- μ j/sample on-chip learning for edge-ai applications. *IEEE Journal of Solid-State Circuits*, 59(8):2717–2729, 2024a.
- Zhang, J., Yin, S., Liu, P., Ying, R., and Wen, F. Curvature-aware zeroth-order optimization for memory-efficient test-time adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

Zhang, Y., Li, P., Hong, J., Li, J., Zhang, Y., Zheng, W., Chen, P.-Y., Lee, J. D., Yin, W., Hong, M., Wang, Z., Liu, S., and Chen, T. Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: A benchmark. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024b.

Zheng, H., Wu, Y., Deng, L., Hu, Y., and Li, G. Going deeper with directly-trained larger spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 11062–11070, 2021.

Zhou, C., Zhang, H., Zhou, Z., Yu, L., Huang, L., Fan, X., Yuan, L., Ma, Z., Zhou, H., and Tian, Y. Qkformer: Hierarchical spiking transformer using qk attention. *Advances in Neural Information Processing Systems*, 37: 13074–13098, 2024.

Zhou, Z., Zhu, Y., He, C., Wang, Y., Yan, S., Tian, Y., and Yuan, L. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations*, 2023.

The Appendix is organized as follows.

Appendix A provides a detailed Related Work, including SNN learning methods, biologically plausible methods, BP-free methods for forward-only learning, subspace-based learning methods, subspace-based BP-free methods in LLM finetuning, and neuromorphic on-chip learning hardware.

Appendix B introduces the advantage of forward-only SZO for on-chip hardware implementation.

Appendix C provides more details of the proposed SZO algorithm.

Appendix D presents more detailed results in the training from scratch experiment.

Appendix E provides more detailed results in the unsupervised model adaptation experiment.

Appendix F compares the ZO gradient estimation variance on SNN and ANN, and shows the effectiveness of the subspace strategy in reducing the ZO estimation variance.

Appendix G compares SZO with other bio-plausible methods.

Appendix H compares the GPU runtime memory usage between SZO and BP with gradient checkpointing.

Appendix I provides more ablation experiments.

Appendix J provides more implementation details of the experiments.

Appendix K provides the proof of Proposition 3.1.

Appendix L provides the proof of Lemma 5.3.

Appendix M provides the proof of Theorem 5.4.

A. Related Work

SNN Learning Methods. The high energy efficiency of SNNs has inspired extensive researches (Maass, 1997; Zenke & Ganguli, 2018; Lee et al., 2016; Rueckauer et al., 2017; Wu et al., 2018; Neftci et al., 2019; Lee et al., 2020a; Fang et al., 2021; Radhakrishnan et al., 2021; Zheng et al., 2021; Han et al., 2020; Rathi & Roy, 2021). A central challenge in training large SNN models lies in the non-differentiability of spiking activations. Early approaches using ANN-to-SNN conversion have demonstrated promising performance on large models by leveraging pre-trained artificial neural networks (Hu et al., 2023; Wu et al., 2022; Pérez-Carrasco et al., 2013; Han et al., 2020; Bu et al., 2022). More recently, a growing body of work has focused on directly training using surrogate gradients with BPTT, which successfully narrow the performance gap between SNNs and ANNs on complex tasks and larger models (Meng et al., 2023; Hu et al., 2021; Li et al., 2021; Yin et al., 2021; Xiao et al., 2022; Zhang & Zhang, 2024; Fang et al., 2023b; Zhou et al., 2023; Yao et al., 2024; Zhou et al., 2024). To address the high complexity of BPTT, approximate methods have been proposed (Bellec et al., 2020; Bohnstingl et al., 2022; Xiao et al., 2022; Meng et al., 2023; Yin et al., 2023).

Biologically Plausible Methods. BP is considered biologically implausible due to its weight transport and update locking problems (Lillicrap et al., 2020). Feedback alignment (Lillicrap et al., 2016) and its variants (Nøkland, 2016; Launay et al., 2020; Frenkel et al., 2021; Liao et al., 2016; Xiao et al., 2019; Akroun et al., 2019) have been proposed to relax the requirement of exact weight symmetry. Target propagation (Lee et al., 2015) provides an alternative by propagating local targets instead of gradients. Equilibrium propagation adopts a two-phase energy-based learning scheme that contrasts free and nudged equilibrium states to obtain local weight updates (Scellier & Bengio, 2017), and has been extended to SNN (Martin et al., 2021). Hebbian learning has long been viewed as a biologically grounded mechanism for synaptic plasticity (Von der Malsburg, 1973; Kohonen, 1982; Oja, 1982; Song & Abbott, 2001; Markram et al., 1997; Bi & Poo, 1998; Gu, 2002; Magee & Grienberger, 2020; Krotov & Hopfield, 2019), which updates synaptic weights based solely on local pre- and post-synaptic activity. Perturbation-based methods offer another biologically plausible approach by estimating learning signals through perturbations (Mazzoni et al., 1991; Seung, 2003; London et al., 2010; Fiete et al., 2007; Miconi, 2017; Bouvier et al., 2018; Aronov et al., 2008; Ali et al., 2013; Flower & Jabri, 1992; Hiratani et al., 2022), which directly send loss differences back to weights and thus do not require any backward passes and weights. However, these methods typically face scalability challenges when applied to large models or complex tasks.

BP-Free Methods for Forward-Only Learning. To avoid the problems of BP, such as update-locking and high memory requirement, BP-free methods, such as the forward gradient (FG) and ZO optimization methods, have recently been shown to be utilizable for neural network training. The FG methods compute gradients based on the directional derivatives that can be computed efficiently via forward differentiation mode (Baydin et al., 2022; Ren et al., 2023; Bacho & Chu, 2024; Fournier et al., 2023). ZO methods not only avoid the backward gradient computation but also can largely reduce the

memory overhead (Chen et al., 2024). Particularly, it has been recently shown that ZO optimization can effectively finetune large language models (Malladi et al., 2023; Zhang et al., 2024b; Guo et al., 2025) and vision Transformers (Zhang et al., 2026).

Subspace-Based Learning. The low-dimensional structure underlying neural network optimization has been well recognized (Li et al., 2018; Larsen et al., 2022; Gur-Ari et al., 2018; Gressmann et al., 2020; Li et al., 2022), which show that optimization trajectories often reside within intrinsic low-dimensional subspaces. Moreover, recent approaches have exploited low-rank structures to enable efficient fine-tuning of large models through low-rank representation (Hu et al., 2022).

Subspace-based BP-Free Methods in LLM finetuning. Current applications of subspace methods combined with ZO optimization are predominantly found in the finetuning of LLMs. For instance, while (Malladi et al., 2023) did not propose a subspace-based ZO optimization algorithm, their theoretical analysis under the Polyak-Łojasiewicz (PL) condition derived a convergence rate related to the effective rank. Several other works, including (Jin et al., 2024; Chen et al., 2025b; Yu et al., 2025), have combined ZO methods with subspace techniques like LoRA, or used random projections to construct the subspace ZO approach.

Neuromorphic On-Chip Learning Hardware. Recent years have seen growing interest in neuromorphic on-chip learning. Early mixed-signal chips, such as BrainScaleS (Schemmel et al., 2010; Billaudelle et al., 2020) and ROLLS (Qiao et al., 2015), use STDP-like mechanisms for local synaptic updates. Digital neuromorphic systems, such as SpiNNaker (Furber et al., 2014), Loihi (Davies et al., 2018), and many others (Baek et al., 2019; Frenkel et al., 2018; 2019; Heidarpur et al., 2019), support programmable plasticity and on-chip learning based on STDP with hardware-efficient designs. Despite their hardware friendliness, STDP-based methods are difficult to scale to deep models required for real-world tasks. Accelerators (Liang et al., 2021; Frenkel & Indiveri, 2022) and (Park et al., 2019; Chen et al., 2025a; Guo et al., 2022; Zhang et al., 2024a; Lee et al., 2020b) explore approximate or local learning to mitigate BPTT overhead.

B. Advantage of Forward-Only SZO for On-Chip Hardware Implementation

It is worth noting that when considering on-chip hardware implementation, forward-only methods are much more advantageous than the BP relied methods. For example, some existing on-chip learning neuromorphic processors that rely on backward computation (e.g., H2learn (Liang et al., 2021), ReckOn (Frenkel & Indiveri, 2022), (Lee et al., 2020b), EPOC (Chen et al., 2025a)) require dedicated feedback circuits to support backward flow, which increases hardware complexity, power consumption, and area cost. In contrast, as a forward-only method, SZO eliminates the need for any feedback pathway, which facilitates compact on-chip implementation and makes it a highly practical solution for power- and area-constrained conditions. While BrainScaleS (Schemmel et al., 2010), ROLLS (Qiao et al., 2015), SpiNNaker (Furber et al., 2014), Loihi (Davies et al., 2018), and many others neuromorphic processors (Baek et al., 2019; Frenkel et al., 2018; 2019; Heidarpur et al., 2019), support local plasticity rules (e.g., STDP) based forward-only on-chip learning, these methods are difficult to scale to deep models required for real-world tasks.

Our target use cases involve resource-constrained on-chip learning, where forward-only rules directly translate to simpler and more energy-efficient hardware designs. More specifically, BP is difficult to implement on resource-constrained hardware due to:

- (1) **The need for dedicated feedback circuits** for error backpropagation.
- (2) **The update-locking problem**, where forward computation must wait for backward computation.
- (3) **High memory overhead** from storing activations, especially as batch-size and time-steps increase, and inefficient weight transport.
- (4) **Inefficient weight transport**, involving hardware-inefficient weight transfer during backward passes.

In contrast, SZO relies only on forward computation and offers strong suitability for practical on-chip learning:

- (1) **No dedicated feedback circuits are required**, as SZO relies only on forward computation.
- (2) **Learning in a low-dimensional subspace enables significantly faster convergence** compared to full-space BP, which is particularly desirable for real-time on-chip learning.
- (3) Although SZO requires storing a subspace matrix of size $d \times k$, it eliminates the need to store activations, thus has **much lower memory overhead than BP** (Figure 5).

- (4) Unlike vanilla ZO methods that require repeatedly sampling high-dimensional random vectors of size d , SZO only samples within a subspace of dimension k (typically tens, e.g., $k = 20, 60$ vs. $d = 11.22\text{M}$ and 25.56M for Spike-ResNet18 and Spike-ResNet50). This dimensionality reduction not only accelerates optimization but also **removes the hardware bottleneck of generating and manipulating extremely high-dimensional random perturbations**, which makes ZO learning substantially more hardware-efficient and practically deployable on neuromorphic chips.

Achieving brain-like neuromorphic on-chip learning has much potential for energy-efficient edge AI with adaptive intelligence, we hope this work inspires future research on efficient and scalable algorithms toward this direction.

C. Details of the Subspace-Based zeroth-Order (SZO) Training Algorithm

A detailed sketch of SZO is presented in Algorithm 1.

Algorithm 1 Subspace-based zeroth-order (SZO) training

```

Initial weight  $\theta_0$ , loss  $\mathcal{L}$ , perturbation scale  $\mu$ , maximum iteration  $T$ , projection matrix  $P = [v_1, v_2, \dots, v_k]$ , and dataset  $\mathcal{D}$ 
for  $t = 1, \dots, T$  do
    Sample batch  $\mathcal{B}_t \subset \mathcal{D}$ 
    Initialize the gradient  $\hat{g}_t \leftarrow 0 \in \mathbb{R}^d$ 
    if use SZO-CGE then
        for  $i = 1, \dots, k$  do
             $\mathcal{L}^+ \leftarrow \mathcal{L}_{\mathcal{B}_t}(\theta_t + \mu v_i)$ 
             $\mathcal{L}^- \leftarrow \mathcal{L}_{\mathcal{B}_t}(\theta_t - \mu v_i)$ 
             $\hat{g}_t \leftarrow \hat{g}_t + \frac{\mathcal{L}^+ - \mathcal{L}^-}{2\mu} v_i$ 
        end for
    else if use SZO-RGE then
        for  $i = 1, \dots, q$  do
            Sample  $z_i \sim \mathcal{N}(0, I_k)$ 
             $\mathcal{L}^+ \leftarrow \mathcal{L}_{\mathcal{B}_t}(\theta_t + \mu P z_i)$ 
             $\mathcal{L}^- \leftarrow \mathcal{L}_{\mathcal{B}_t}(\theta_t - \mu P z_i)$ 
             $\hat{g}_t \leftarrow \hat{g}_t + \frac{1}{q} \frac{\mathcal{L}^+ - \mathcal{L}^-}{2\mu} P z_i$ 
        end for
    end if
    Updated model  $\theta_t$  through SGD with momentum  $\theta_{t+1} = \theta_t - \eta \hat{g}_t + \beta(\theta_t - \theta_{t-1})$ .
end for
    
```

D. Detailed Results in the Training from Scratch Experiment

Figure 7 and Figure 8 show the test accuracy curves of the BP-subspace and SZO-CGE/RGE methods in the training procedure on CIFAR-100 (Spike-ResNet18) and ImageNet1K (Spike-ResNet50), respectively.

E. Detailed Results in the Unsupervised Model Adaptation Experiment

We provide detailed results in the unsupervised model adaptation experiment in Table 5 to Table 10, in which the accuracy for each of the 15 corruptions are reported for the compared methods.

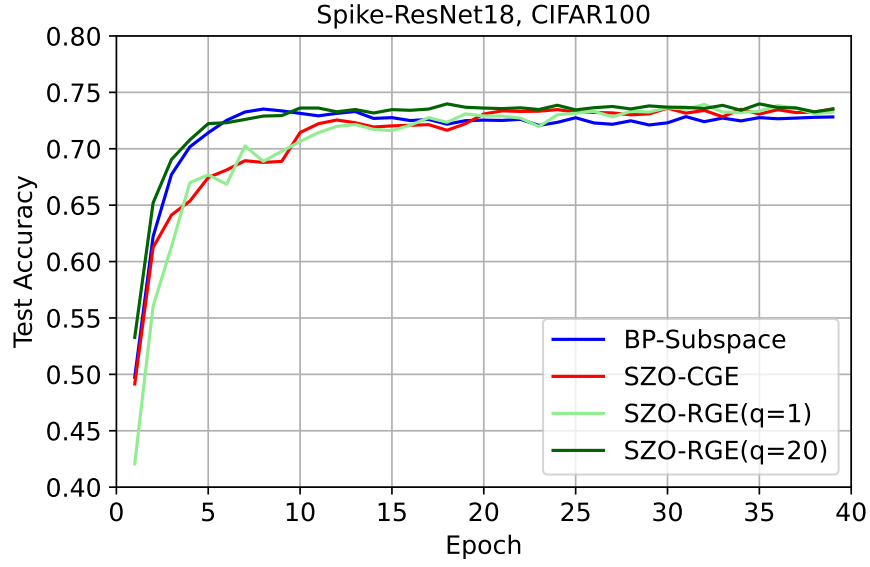


Figure 7. Test accuracy of Spike-ResNet18 on CIFAR100.

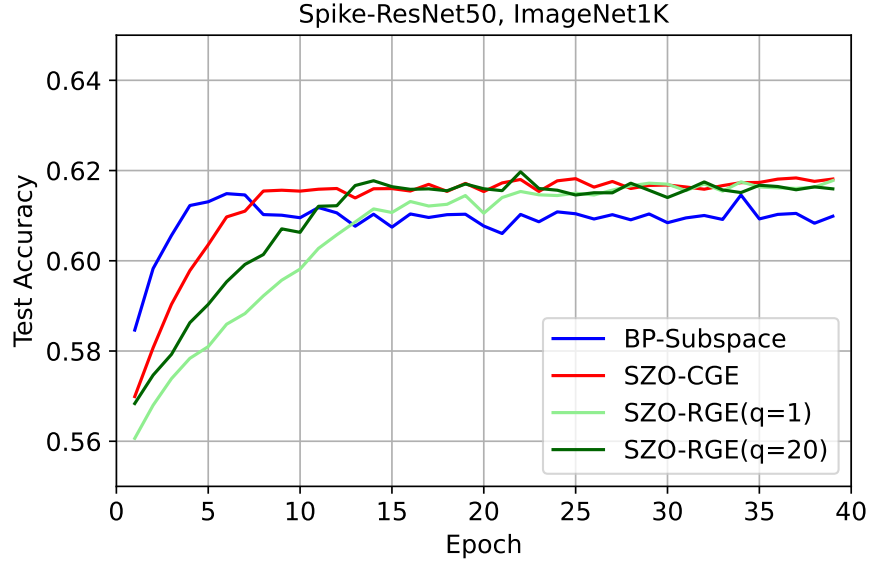


Figure 8. Test accuracy of Spike-ResNet50 on ImageNet1k.

Table 5. Accuracy (%) on CIFAR10C dataset with corruption severity level 5 under *Reset* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	Acc.
NoAdapt	X	14.57	17.47	24.49	50.88	31.54	52.10	53.67	55.79	42.39	53.70	80.56	22.48	67.07	36.55	71.85	45.01
BN	X	53.79	54.13	57.78	73.37	56.85	71.00	74.20	62.45	61.89	68.67	79.24	61.87	70.01	66.22	71.43	65.53
Tent	✓	54.88	55.96	58.57	74.01	57.58	71.33	75.21	63.21	62.73	69.46	79.18	61.59	70.22	66.91	71.33	66.14
BP-Subspace	✓	67.45	67.24	64.58	74.15	58.31	72.41	75.45	70.23	69.75	70.52	82.35	24.34	71.27	71.78	75.45	67.69
SZO-CGE	X	70.97	73.94	65.70	75.19	44.49	68.32	75.47	70.10	71.08	70.04	83.50	19.33	70.03	72.65	75.52	67.09
SZO-RGE(q=10)	X	62.10	52.59	56.59	62.30	35.98	58.71	68.54	60.80	49.00	52.67	77.66	28.23	61.09	57.29	72.80	57.09

Table 6. Accuracy (%) on CIFAR100C dataset with corruption severity level 5 under *Reset* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average Acc.
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	
NoAdapt	X	8.01	9.55	9.71	29.85	21.51	38.16	36.05	41.89	28.72	32.53	59.36	10.39	51.91	22.56	46.91	29.81
BN	X	38.06	37.71	38.06	62.28	45.21	58.92	63.06	49.52	49.82	51.76	64.34	53.77	56.12	55.36	51.83	51.72
Tent	✓	39.22	39.36	39.71	62.92	45.76	59.42	63.77	50.44	50.09	52.22	65.14	53.85	56.60	56.00	52.21	52.45
BP-Subspace	✓	75.69	75.93	49.68	86.19	56.32	76.52	83.19	76.20	80.02	68.52	91.56	67.28	72.93	84.23	81.06	75.02
SZO-CGE	X	74.22	73.65	48.36	81.65	52.60	74.29	79.81	72.88	77.41	65.78	90.98	66.43	71.03	79.54	78.46	72.47
SZO-RGE(q=10)	X	57.79	59.13	47.89	70.32	52.00	67.21	71.02	62.27	63.48	59.83	75.45	61.37	63.98	68.62	65.11	63.03

 Table 7. Accuracy (%) on ImageNetC dataset with corruption severity level 5 under *Reset* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average Acc.
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	
NoAdapt	X	0.62	0.88	0.78	7.44	6.78	8.52	16.02	5.02	7.82	5.92	34.32	1.12	18.58	8.02	18.62	9.36
BN	X	5.82	7.40	6.02	11.80	13.10	17.54	28.38	16.42	19.92	28.40	48.20	9.16	39.58	36.16	30.10	21.20
Tent	✓	7.24	8.76	8.10	11.94	13.80	19.90	30.08	17.18	21.14	31.76	49.14	8.84	42.78	38.62	33.00	22.82
BP-Subspace	✓	45.64	37.64	33.00	14.98	18.65	23.12	33.45	19.36	25.98	30.00	50.26	5.85	45.77	46.18	46.53	31.76
SZO-CGE	X	50.55	40.80	35.56	15.62	19.65	22.87	33.80	19.69	27.15	29.67	50.28	6.60	46.45	46.98	48.37	32.94
SZO-RGE(q=10)	X	47.92	38.998	33.93	14.85	18.83	22.73	33.08	19.88	26.49	30.50	50.54	6.64	46.17	47.24	47.34	32.34

 Table 8. Accuracy (%) on CIFAR10C dataset with corruption severity level 5 under *Continual* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average Acc.
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	
NoAdapt	X	14.57	17.47	24.49	50.88	31.54	52.10	53.67	55.79	42.39	53.70	80.56	22.48	67.07	36.55	71.85	45.01
BN	X	53.79	54.13	57.78	73.37	56.85	71.00	74.20	62.45	61.89	68.67	79.24	61.87	70.01	66.22	71.43	65.53
Tent	✓	55.83	58.65	61.21	74.04	57.84	70.45	73.76	63.45	62.45	66.81	74.93	55.56	65.54	62.54	65.70	64.58
BP-Subspace	✓	70.48	75.57	69.91	78.40	63.50	75.59	78.57	75.54	76.32	73.43	85.67	45.31	66.60	76.32	79.69	72.73
SZO-CGE	X	74.09	79.31	70.74	80.14	64.68	76.17	80.52	76.10	77.06	75.44	86.43	66.60	75.84	78.02	80.29	76.10
SZO-RGE(q=10)	X	59.20	64.79	63.18	65.29	47.49	37.73	25.36	19.22	14.53	13.10	13.87	10.87	11.40	10.65	9.71	31.09

 Table 9. Accuracy (%) on CIFAR100C dataset with corruption severity level 5 under *Continual* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average Acc.
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	
NoAdapt	X	8.01	9.55	9.71	29.85	21.51	38.16	36.05	41.89	28.72	32.53	59.36	10.39	51.91	22.56	46.91	29.81
BN	X	38.06	37.71	38.06	62.28	45.21	58.92	63.06	49.52	49.82	51.76	64.34	53.77	56.12	55.36	51.83	51.72
Tent	✓	38.51	39.34	40.99	63.34	47.01	60.62	64.66	53.39	52.33	53.61	67.32	54.05	58.22	59.26	54.88	53.84
BP-Subspace	✓	55.37	69.93	59.26	84.31	65.30	80.21	87.03	80.47	84.13	76.04	94.13	81.23	80.60	87.75	86.52	78.15
SZO-CGE	X	62.56	79.04	63.56	88.84	65.79	81.85	86.51	80.61	83.81	75.93	94.05	80.10	79.39	86.66	86.59	79.69
SZO-RGE(q=10)	X	57.79	72.71	59.10	83.88	62.43	78.28	83.21	76.70	79.79	72.31	91.04	75.34	76.94	84.96	83.39	75.86

 Table 10. Accuracy (%) on ImageNetC dataset with corruption severity level 5 under *Continual* setting.

Method	BP	Noise			Blur Defoc.				Weather				Digital				Average Acc.
		Gauss.	Shot	Impul.	Defoc.	Glass	Motion	Zoom	Snow	Frost	Fog	Brit.	Contr.	Elas.	Pix.	JPEG	
NoAdapt	X	0.62	0.88	0.78	7.44	6.78	8.52	16.02	5.02	7.82	5.92	34.32	1.12	18.58	8.02	18.62	9.36
BN	X	5.82	7.40	6.02	11.80	13.10	17.54	28.38	16.42	19.92	28.40	48.20	9.16	39.58	36.16	30.10	21.20
Tent	✓	7.58	10.12	8.76	8.38	6.06	5.54	4.84	1.38	0.86	0.72	1.50	0.38	0.96	0.68	0.60	3.89
BP-Subspace	✓	19.03	33.09	32.93	16.65	20.92	24.34	34.93	22.12	28.48	30.79	51.71	13.92	48.81	50.97	53.51	32.15
SZO-CGE	X	23.30	36.61	36.09	17.20	21.53	24.95	35.55	22.70	29.22	31.09	52.07	14.61	49.55	51.95	54.18	33.37
SZO-RGE(q=10)	X	21.76	35.46	34.98	17.07	21.32	25.03	35.59	23.04	29.05	31.19	51.91	15.07	49.38	52.13	54.36	33.16

F. Comparison of ZO Gradient Estimation Variance on SNN and ANN, and the Effectiveness of the Subspace Method in Variance Reduction

We report the variance of gradient of all parameters estimated by full-space ZO (RGE, $q = 20$) on ANN and SNN with same architecture in Figure 9. It can be seen that ZO method indeed produces higher variance in SNN compared to ANN. This empirically validates the conclusion of Proposition 3.1 that the non-smooth nature of the Heaviside function is a significant source of gradient variance.

Figure 10 and Figure 11 show that the proposed subspace method reduces the variance of ZO gradient estimation for ANN (ReLU) and SNN, respectively. While the subspace strategy also lowers the ZO variance in ANNs, it yields a much larger variance reduction in SNNs, which highlights its particular effectiveness in mitigating the Heaviside-induced instability.

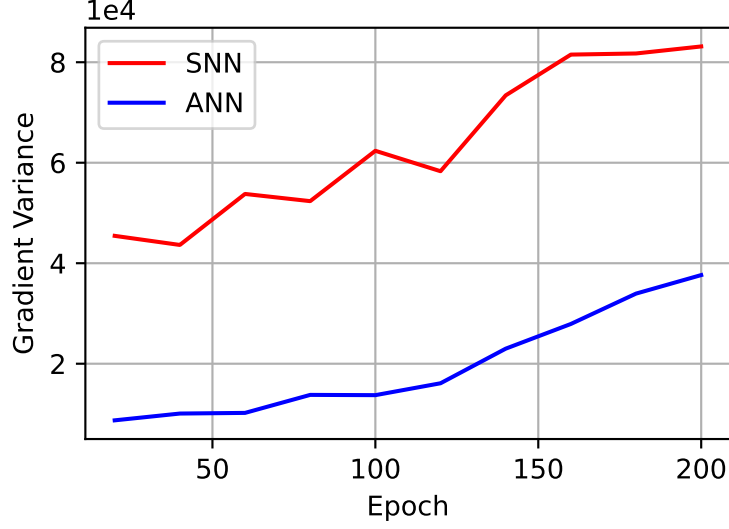


Figure 9. Gradient variance of full-space ZO on CIFAR10 with SNN (SpikeResNet20) and ANN (ResNet20).

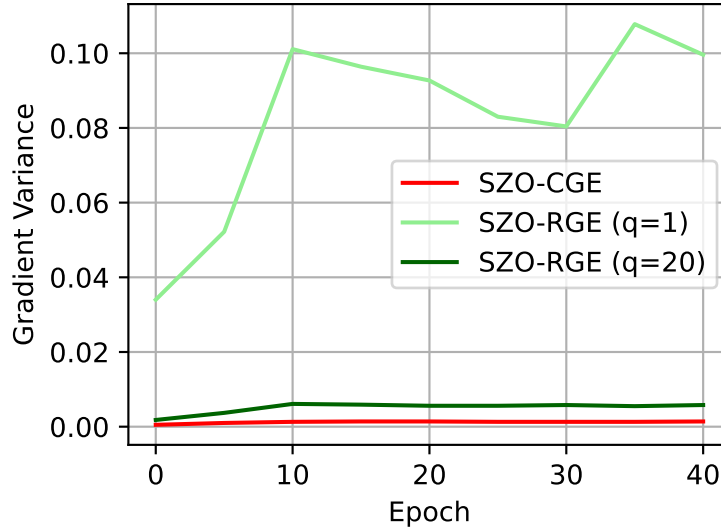


Figure 10. Gradient variance of different methods on CIFAR10 with ANN (ResNet20).

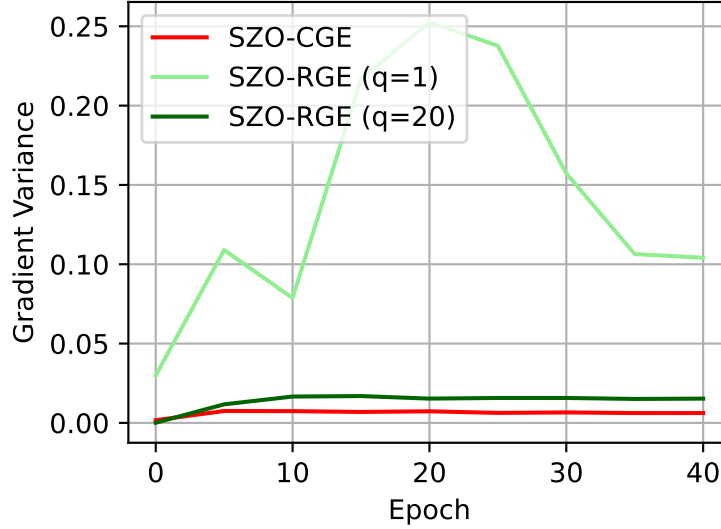


Figure 11. Gradient variance of different methods on CIFAR10 with SNN (SpikeResNet20).

G. Comparison with Other Bio-Plausible Methods

Equilibrium propagation based methods, such as EqSpike (Martin et al., 2021), are based on local energy minimization and provide a BP-free, hardware-friendly learning paradigm. However, these methods have so far been demonstrated on small models and simpler datasets, and their scalability to larger SNNs and more challenging datasets needs further study. We additionally evaluated SZO under the same model and dataset setting as Martin et al. (2021) and compare it with EqSpike. As shown in Table 11, both SZO and EqSpike achieve performance close to BP.

We also conducted experiments to compare with DFA (Nøkland, 2016) and OPZO (Xiao et al., 2026). OPZO is a pseudo-ZO algorithm that also uses direct error feedback to each layer. We consider both the Spike-CNN model in Xiao et al. (2026) and other 3 models in our work. As shown in Table 12, OPZO performs well on shallow CNN and VGG, but is limited on deep ResNets where SZO significantly outperforms OPZO and DFA.

Table 11. Accuracy (%) comparison for training from scratch with EqSpike.

Dataset	Model	BP	BP-Subspace	SZO-CGE	SZO-RGE ($q = 20$)	EqSpike
MNIST	Spike-MLP(784-100-10)	97.57	97.51	97.56	97.50	96.87

Table 12. Accuracy (%) comparison for training from scratch with DFA and OPZO.

Dataset	Model	Params	BP	BP-Subspace	SZO-CGE	SZO-RGE($q = 1$)	SZO-RGE($q = 20$)	OPZO	DFA
CIFAR10	Spike-CNN (5-layer)	3.92M	90.79	90.78	90.78	90.67	90.72	89.42	82.38
CIFAR100	Spike-CNN (5-layer)	4.66M	67.17	67.40	67.42	67.32	67.03	64.77	54.76
CIFAR10	Spike-ResNet20	0.27M	86.74	86.42	86.21	81.29	83.71	54.45	38.79
CIFAR100	Spike-ResNet18	11.22M	74.51	73.32	73.35	73.77	73.79	36.90	28.36
DVS-Gesture	Spike-VGG11	9.50M	97.57	96.65	96.65	91.78	95.72	96.06	91.43

H. Comparison with BP Gradient Checkpointing

We implement BP with gradient checkpointing for SNN models and compared GPU runtime memory usage with SZO.

We use `torch.utils.checkpoint` provided by Pytorch. Directly applying checkpointing to SNN is unsuitable, because it only frees activations by default, while membrane potentials remain stored as internal state and still occupy large GPU memory. We therefore adopt a multi-step scheme, where each layer iterates over all time-steps locally and only passes

the multi-step output to the next layer. This sequential layer-wise computation mode ensures that when using checkpointing, both activations and membrane potentials of non-checkpointed layers are freed after their forward pass and only recomputed when needed during backpropagation.

We compare the GPU runtime memory on ImageNet1K and DVS-Gesture with Spike-ResNet50 and Spike-VGG11 respectively. To comprehensively assess scaling behavior, we evaluate a range of batch-sizes and time-steps. The runtime usage on a single NVIDIA V100 GPU (32 G) is reported in Tables 13–16. OOM denotes out-of-memory on a single V100 GPU with 32 GB memory.

The results show that the gradient checkpointing method significantly reduces the memory footprint of BPTT-based SNN training, which confirms its effectiveness as a strong engineering solution. SZO and BP+checkpoint can be comparable in memory usage for small batch-sizes and small time-steps. However, the memory of BP with gradient checkpointing still grows steeply with both batch-size and time step. In contrast, the memory of SZO increases only marginally as batch-size increases and remains almost invariant to time-step. More importantly, **the gradient checkpointing method still relies on BP and requires dedicated feedback circuits for error backpropagation when implementing on hardware. In comparison, the proposed SZO does not require feedback circuits, as SZO relies only on forward computation, which simplifies the implementation on hardware chips.** This advantage is detailed in Section B.

Table 13. GPU runtime memory (MB) with different batch sizes on ImageNet1K using Spike-ResNet50 (timestep = 4).

Batch size	32	48	64	80	96
BP	14921	22278	29604	OOM	OOM
BP+checkpoint, segment=4	8927	13129	17529	21839	26123
BP+checkpoint, segment=8	7804	10497	13183	15873	18560
SZO	8067	8889	9684	10511	11302

Table 14. GPU runtime memory (MB) with different time-steps on ImageNet1K using Spike-ResNet50 (batch size = 32).

Time step	2	4	6	8	10
BP	9403	14921	22389	29785	OOM
BP+checkpoint, segment=4	9399	8927	13294	17651	22012
BP+checkpoint, segment=8	7557	7804	10501	13199	15903
SZO	8067	8067	8067	8067	8067

Table 15. GPU runtime memory (MB) with different batch sizes on DVS-Gesture using Spike-VGG11 (timestep = 20).

Batch size	4	8	16	32	40
BP	5755	7959	15728	29031	OOM
BP+checkpoint, segment=4	4106	5656	11183	22203	27767
BP+checkpoint, segment=8	4008	5201	10271	20416	25488
SZO	1103	1251	1546	2137	2433

Table 16. GPU runtime memory (MB) with different time-steps on DVS-Gesture using Spike-VGG11 (batch size = 16).

Time step	10	20	30	40	50
BP	7941	15728	23545	29065	OOM
BP+checkpoint, segment=4	5664	11183	16706	22223	27746
BP+checkpoint, segment=8	5233	10271	15313	20351	25391
SZO	1526	1546	1566	1586	1606

I. More Ablation Experiments

Number of queries. We evaluated SZO-RGE under different numbers of queries, with perturbation scale fixed at $\mu = 1$. As shown in Table 17, SZO-RGE can achieve accuracy comparable to SZO-CGE even when q is smaller than the subspace dimension $k = 60$. We further evaluated the computational trade-off of SZO-RGE under different numbers of perturbations q in the training from scratch task. We report the converged epoch, wall-clock converged time, and number of forward passes (#FP) required to reach the maximum accuracy. The results are shown in Table 19. While smaller q yields faster training but lower accuracy, increasing q improves the final accuracy at the cost of significantly longer training time and higher computational overhead.

Comparison of one-point and two-point gradient estimation We have conducted additional experiments using single-point gradient estimation. The results in Table 18 show that one-point estimation can achieve comparable performance to two-point estimation on CIFAR100.

Table 17. Accuracy (%) of SZO-RGE on training from scratch task with different number of queries.

Dataset	Network	$q = 1$	$q = 20$	$q = 40$	$q = 60$
CIFAR10	Spike-ResNet20	81.29 $\pm$ 0.35	83.71 $\pm$ 0.34	83.44 $\pm$ 0.16	83.52 $\pm$ 0.09
CIFAR100	Spike-ResNet18	73.77 $\pm$ 0.26	73.79 $\pm$ 0.22	73.90 $\pm$ 0.24	73.87 $\pm$ 0.17

Table 18. Accuracy (%) comparison of one-point and two-point gradient estimation on training from scratch task.

Dataset	Network	BP	BP-Subspace	SZO-CGE	SZO-CGE (one-point)
CIFAR10	Spike-ResNet20	86.65	86.22	86.01	83.00
CIFAR100	Spike-ResNet18	74.42	73.37	73.42	73.28

Table 19. Comparison of computational overhead in the training-from-scratch task on CIFAR10 (Spike-ResNet20) and CIFAR100 (Spike-ResNet18).

q	CIFAR10 / Spike-ResNet20					CIFAR100 / Spike-ResNet18				
	Converged epoch	Converged time (min)	#FP	#BP	Max Acc(%)	Converged epoch	Converged time (min)	#FP	#BP	Max Acc(%)
$q = 1$	39	22.8	45747	0	80.55	39	60.7	45747	0	73.47
$q = 20$	33	224.3	529023	0	83.96	15	243.2	240465	0	73.55
$q = 40$	24	331.0	760104	0	83.60	21	680.4	665091	0	73.64
$q = 60$	29	621.0	1372019	0	83.61	15	708.6	709665	0	73.68

J. More Implementation Details of the Experiments

All network architectures utilize LIF neurons with $\tau = 1.1$ and $V_{th} = 1$. A triangle surrogate function is used in BPTT. Furthermore, we keep the running statistics of batch norm layers before each perturbation on training step t the same. We run each method 5 times independently with different random seeds and report the results as mean $\pm$ standard deviation.

In the training from scratch experiment, the model vectors used to construct the subspace are sampled from the BPTT training trajectory. During the BPTT training stage, the SGD optimizer is used for all datasets, with an initial learning rate of 0.1, momentum of 0.9, and a weight decay of 1e-4. The learning rate is decayed using a cosine method.

The SGD optimizer is also utilized in BP-Subspace and SZO for the training from scratch and continual training tasks, but the initial learning rate is set to 1, with momentum at 0.9 and no weight decay. In the SZO unsupervised adaptation experiment, a constant learning rate of 1 and 0.1 is used for *Reset* setting and *Continual* setting respectively.

The batch size is set to 128 for CIFAR10 and CIFAR100, 16 for DVS-Gesture, and 512 for ImageNet1K. For adaptation on corrupted datasets, the batch size is 100 for CIFAR10C and CIFAR100C, and 500 for ImageNetC. Common data augmentation methods, including random cropping and random horizontal flipping, are applied to static datasets. For DVS-Gesture, we integrate events and split each sample into 20 frames for the input of each time step.

K. Proof of Proposition 3.1

Proof. First, we consider the Heaviside function $f_h(z) = \mathbb{I}(z > 0)$. Here, without loss of generality, we assume the firing threshold to be zero. From the symmetry and continuity assumption in Proposition 1, we have $\Pr(z > 0) = \Pr(z < 0) = \frac{1}{2}$. Hence the output of the Heaviside function is a Bernoulli distribution, i.e., $f_h(z) \sim \text{Bernoulli}(\frac{1}{2})$. Then, it follows that

$$\text{Var}(f_h(z)) = \frac{1}{4}. \quad (7)$$

Then, we consider the sigmoid function $f_s(z) = \frac{1}{1+e^{-z}}$. Using a first-order Taylor expansion around $z = 0$ we have $f_s(z) = \frac{1}{2} + \frac{1}{4}z + O(z^2)$. Then, we have

$$\text{Var}(f_s(z)) = \frac{1}{16}\text{Var}(z) + O(\sigma^4) = \frac{\sigma^2}{16} + o(\sigma^2). \quad (8)$$

Next, we consider the ReLU function $f_r(z) = \max\{0, z\}$. From the symmetry assumption in Proposition 1, we have

$$\mathbb{E}[f_r(z)] = \frac{1}{2}\mathbb{E}[|z|],$$

and

$$\mathbb{E}[f_r^2(z)] = \frac{1}{2}\mathbb{E}[z^2] = \frac{1}{2}\sigma^2.$$

Thus, it follows that

$$\text{Var}(f_r(z)) = \frac{\sigma^2}{2} - \frac{1}{4}(\mathbb{E}[|z|])^2. \quad (9)$$

Then, from the Cauchy–Schwarz inequality, we have $\mathbb{E}[|z|] \leq \sigma$. Hence, we have

$$\frac{\sigma^2}{4} \leq \text{Var}(f_r(z)) \leq \frac{\sigma^2}{2}. \quad (10)$$

Finally, comparing across (7), (8) and (10) gives the final result. $\square$

L. Proof of Lemma 5.3

To obtain the estimation error, we first give the following lemma about the form of the gradient of the smoothed loss.

Lemma L.1 (Gradient of the smooth loss). *Given any $\theta \in \mathbb{R}^d$ and data $x \sim \mathcal{D}$, the gradient of the smoothed loss respect to parameter θ has the following explicit form*

$$\nabla \mathcal{L}_x^\mu(\theta) = \frac{1}{\mu} \mathbb{E}_{u \sim \mathcal{N}(0, I_d)} [\mathcal{L}_x(\theta + \mu u) \cdot u].$$

Proof. We first define a new variable $v = \theta + \mu u$ and defined its domain as $\Omega' = \{v | v = \theta + \mu u \text{ for all } u \in \Omega\}$. Because $u \sim \mathcal{N}(0, I_d)$, we have $v \sim \mathcal{N}(\theta, \mu^2 I_d)$. We denote $p(\cdot; \theta, \mu)$ as the density function of $\mathcal{N}(\theta, \mu^2 I_d)$. So from the definition of the gradient of the smoothed loss, we have

$$\begin{aligned} \nabla \mathcal{L}_x^\mu(\theta) &= \nabla \mathbb{E}_{u \sim \mathcal{N}(0, I_d)} [\mathcal{L}_x(\theta + \mu u)] \\ &= \nabla \mathbb{E}_{v \sim \mathcal{N}(\theta, \mu^2 I_d)} [\mathcal{L}_x(v)] \\ &= \nabla \int_{\Omega'} \mathcal{L}_x(v) p(v; \theta, \mu) dv \\ &= \int_{\Omega'} \mathcal{L}_x(v) \nabla p(v; \theta, \mu) dv \\ &= \int_{\Omega'} \mathcal{L}_x(v) p(v; \theta, \mu) \nabla \log p(v; \theta, \mu) dv. \end{aligned}$$

From the definition of $p(\cdot; \theta, \mu)$, its log function is

$$\log p(v; \theta, \mu) = \log \left(\frac{1}{(2\pi\mu^2)^{d/2}} \exp \left(-\frac{\|v - \theta\|^2}{2\mu^2} \right) \right) = -\frac{\|v - \theta\|^2}{2\mu^2} - \frac{d}{2} \log(2\pi\mu^2).$$

So the derivative of this log function is

$$\nabla \log p(v; \theta, \mu) = \frac{v - \theta}{\mu^2}.$$

Finally, the gradient can be obtained as

$$\begin{aligned} \nabla \mathcal{L}_x^\mu(\theta) &= \int_{\Omega} \mathcal{L}_x(v) p(v; \theta, \mu) \frac{v - \theta}{\mu^2} dv \\ &= \frac{1}{\mu^2} \mathbb{E}_{v \sim \mathcal{N}(\theta, \mu^2 I_d)} [\mathcal{L}_x(v) \cdot (v - \theta)] \\ &= \frac{1}{\mu^2} \mathbb{E}_{u \sim \mathcal{N}(0, I_d)} [\mathcal{L}_x(\theta + \mu u) \cdot (\theta + \mu u - \theta)] \\ &= \frac{1}{\mu} \mathbb{E}_{u \sim \mathcal{N}(0, I_d)} [\mathcal{L}_x(\theta + \mu u) \cdot u]. \end{aligned}$$

The proof is completed. $\square$

Proof of Lemma 5.3. Using the bias-variance decomposition and denote the random vector for SZO as w , we have

$$\begin{aligned} &\mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}}^\mu(\theta_t) \right\|^2 \right] \\ &= \mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] + \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] - \nabla \mathcal{L}_{\mathcal{B}}^\mu(\theta_t) \right\|^2 \right] \\ &= \mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] \right\|^2 + \left\| \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] - \nabla \mathcal{L}_{\mathcal{B}}^\mu(\theta_t) \right\|^2 \right] \end{aligned} \quad (11)$$

We denote $g_i = \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu P w_i) - \mathcal{L}_{\mathcal{B}}(\theta - \mu P w_i)}{2\mu} P w_i$, then the variance term can be bounded with Assumption 5.1 as below

$$\begin{aligned} &\mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] \right\|^2 \right] \\ &\stackrel{(a)}{=} \frac{1}{q} \mathbb{E}_{w_1 \sim \mathcal{N}(0, I_k)} \left[\|g_1 - \mathbb{E}[g_1]\|^2 \right] \\ &\stackrel{(b)}{\leq} \frac{1}{q} \mathbb{E}_{w_1 \sim \mathcal{N}(0, I_k)} \left[\|g_1\|^2 \right] \\ &= \frac{1}{q} \mathbb{E}_{w_1 \sim \mathcal{N}(0, I_k)} \left[\left\| \frac{\mathcal{L}_{\mathcal{B}}(\theta + \mu P w_1) - \mathcal{L}_{\mathcal{B}}(\theta - \mu P w_1)}{2\mu} P w_1 \right\|^2 \right] \\ &\stackrel{(c)}{\leq} \frac{M^2}{q\mu^2} \mathbb{E}_{w_1 \sim \mathcal{N}(0, I_k)} \left[\|P w_1\|^2 \right], \end{aligned}$$

where the equation (a) holds because $g_1, \dots, g_q$ are independent random variables, so the property of variance is $\text{Var}(\frac{1}{q} \sum_i g_i) = \frac{1}{q^2} \sum_i \text{Var}(g_i) = \frac{1}{q} \text{Var}(g_1)$, the inequality (b) holds because of the property of variance $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, inequality (c) holds because of the Assumption 5.1. The bound of $\mathbb{E}_{w_1 \sim \mathcal{N}(0, I_k)} [\|P w_1\|^2]$ can be further calculated as

$$\mathbb{E} [\|P w_1\|^2] = \mathbb{E} [w_1^\top P^\top P w_1] = \mathbb{E} [\text{Tr}(w_1^\top P^\top P w_1)] = \text{Tr}(P^\top P \mathbb{E}[w_1^\top w_1]) = \text{Tr}(I_k I_k) = k$$

So the variance term can be bounded as

$$\mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \mathbb{E} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t)] \right\|^2 \right] \leq \frac{M^2 k}{q\mu^2} \quad (12)$$

Using the inequality $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ and Assumption 5.2, the bias term satisfies the following inequality

$$\begin{aligned} \left\| \mathbb{E} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) \right] - \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 &\leq 2 \left\| \mathbb{E} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) \right] - PP^{\top} \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 \\ &\quad + 2 \left\| PP^{\top} \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 \\ &\leq 2 \left\| \mathbb{E} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) \right] - PP^{\top} \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 + 2\varepsilon^2. \end{aligned} \quad (13)$$

Following Lemma L.1, the first term in the RHS of equation (13) has the following relation

$$\begin{aligned} &\left\| \mathbb{E} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) \right] - PP^{\top} \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 \\ &= \left\| \mathbb{E}_{w_1} [g_1] - PP^{\top} \frac{1}{\mu} \mathbb{E}_u [\mathcal{L}_{\mathcal{B}}(\theta + \mu u) \cdot u] \right\|^2 \\ &= \left\| \frac{1}{\mu} \mathbb{E}_{w_1} [\mathcal{L}_{\mathcal{B}}(\theta + \mu Pw_1) \cdot Pw_1] - \frac{1}{\mu} \mathbb{E}_u [\mathcal{L}_{\mathcal{B}}(\theta + \mu u) \cdot PP^{\top} u] \right\|^2. \end{aligned}$$

We define two new random variable $u_{\parallel} = PP^{\top} u$ and $u_{\perp} = (I - PP^{\top})u$. Next, we will show that $u_{\parallel}$ and $u_{\perp}$ are two independent variables and $u_{\parallel} \sim \mathcal{N}(0, PP^{\top})$, which is the same as Pw_1 . From the definition of the two variables, their covariance matrix is

$$\begin{aligned} \text{Cov}(u_{\parallel}, u_{\perp}) &= \mathbb{E} [(u_{\perp} - \mathbb{E}[u_{\perp}])(u_{\parallel} - \mathbb{E}[u_{\parallel}])^{\top}] = \mathbb{E} [u_{\parallel}^{\top} u_{\perp}] \\ &= \mathbb{E} [PP^{\top} u ((I - PP^{\top})u)^{\top}] = \mathbb{E} [PP^{\top} uu^{\top} (I - PP^{\top})^{\top}] \\ &= PP^{\top} \mathbb{E} [uu^{\top}] (I - PP^{\top})^{\top} = PP^{\top} I_d (I - PP^{\top})^{\top} = PP^{\top} I_d (I - PP^{\top}) \\ &= PP^{\top} - PP^{\top} PP^{\top} = PP^{\top} - PI_k P^{\top} = 0 \end{aligned}$$

So we prove that $u_{\parallel}$ and $u_{\perp}$ are independent variable. Because $\mathbb{E} [u_{\parallel}]$ is clearly zero, we only need to calculate its covariance matrix

$$\begin{aligned} \text{Cov}(u_{\parallel}, u_{\parallel}) &= \mathbb{E} [u_{\parallel} u_{\parallel}^{\top}] = \mathbb{E} [PP^{\top} u (PP^{\top} u)^{\top}] = \mathbb{E} [PP^{\top} uu^{\top} (PP^{\top})^{\top}] \\ &= PP^{\top} \mathbb{E} [uu^{\top}] (PP^{\top})^{\top} = PP^{\top} I_d (PP^{\top})^{\top} = PP^{\top} I_d PP^{\top} \\ &= PP^{\top} PP^{\top} = PI_k P^{\top} = PP^{\top}. \end{aligned}$$

The mean of Pw_1 is clearly zero, and the covariance matrix for Pw_1 is

$$\text{Cov}(Pw_1, Pw_1) = \mathbb{E} [Pw_1 (Pw_1)^{\top}] = \mathbb{E} [Pw_1 w_1^{\top} P^{\top}] = PI_k P^{\top} = PP^{\top}.$$

So we proved that Pw_1 and $u_{\parallel}$ are sample from a same distribution. So the first term in the RHS of equation (13) can be bounded as

$$\begin{aligned} &\left\| \frac{1}{\mu} \mathbb{E}_{w_1} [\mathcal{L}_{\mathcal{B}}(\theta + \mu Pw_1) \cdot Pw_1] - \frac{1}{\mu} \mathbb{E}_u [\mathcal{L}_{\mathcal{B}}(\theta + \mu u) \cdot PP^{\top} u] \right\|^2 \\ &= \left\| \frac{1}{\mu} \mathbb{E}_{w_1} [\mathcal{L}_{\mathcal{B}}(\theta + \mu Pw_1) \cdot Pw_1] - \frac{1}{\mu} \mathbb{E}_{w_1} [\mathbb{E}_u [\mathcal{L}_{\mathcal{B}}(\theta + \mu(u_{\parallel} + u_{\perp}))] \cdot Pw_1] \right\|^2 \\ &= \left\| \frac{1}{\mu} \mathbb{E}_{w_1} [\mathcal{L}_{\mathcal{B}}(\theta + \mu Pw_1) - \mathbb{E}_u [\mathcal{L}_{\mathcal{B}}(\theta + \mu(u_{\parallel} + u_{\perp}))]] \cdot Pw_1 \right\|^2 \\ &\leq \frac{4M^2}{\mu^2} \|\mathbb{E}_{w_1} [Pw_1]\|^2 \leq \frac{4M^2}{\mu^2} \mathbb{E}_{w_1} [\|Pw_1\|^2] \\ &\leq \frac{4M^2 k}{\mu^2} \end{aligned} \quad (14)$$

Combining (11), (12), (13) and (14), with $q \geq 1$, we can have the final result as

$$\mathbb{E}_{w \sim \mathcal{N}(0, I_k)} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}}^{\mu}(\theta_t) \right\|^2 \right] \leq \frac{M^2 k}{q\mu^2} + \frac{8M^2 k}{\mu^2} + 2\varepsilon^2 \leq \frac{9M^2 k}{\mu^2} + 2\varepsilon^2.$$

The proof is completed. $\square$

M. Proof of Theorem 5.4

To analyze the convergence rate in the presence of momentum, we quantify the one-step difference between the gradient evaluated at the parameter without momentum, θ_t , and the gradient evaluated at the momentum-adjusted parameter, m_t .

Lemma M.1 (One-step momentum difference). *Suppose the smoothed loss function $\mathcal{L}^\mu$ and the projection matrix P satisfy Assumptions 5.1 and 5.2, and let $\mathcal{B}$ be a mini-batch data of size B . Under the gradient estimator (5) and parameter update rule (6), the following bound holds:*

$$\|\nabla \mathcal{L}^\mu(m_t) - \nabla \mathcal{L}^\mu(\theta_t)\|^2 \leq \frac{L\beta^2\eta^2}{(1-\beta)^4} \left(2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2k}{\mu^2} + 4\epsilon^2 \right).$$

Proof. To facilitate the analysis, we further define $p_t = \frac{\beta}{1-\beta}(\theta_t - \theta_{t-1})$ and $m_t = \theta_t + p_t$. This yields the following relation:

$$\theta_{t+1} + p_{t+1} = \theta_t + p_t - \frac{\eta}{1-\beta} \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta) \Rightarrow m_{t+1} = m_t - \frac{\eta}{1-\beta} \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta).$$

Because the smoothed loss function $\mathcal{L}^\mu(\theta)$ is L -smooth respective to θ , then we have

$$\|\nabla \mathcal{L}^\mu(m_t) - \nabla \mathcal{L}^\mu(\theta_t)\|^2 \leq L^2 \|m_t - \theta_t\|^2 = L^2 \|p_t\|^2 = \frac{L^2\beta^2}{(1-\beta)^2} \|\theta_t - \theta_{t-1}\|^2$$

Then from the definition of $\theta_t - \theta_{t-1}$, we have

$$\begin{aligned} \theta_t - \theta_{t-1} &= \beta(\theta_{t-1} - \theta_{t-2}) - \eta \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_{t-1}) \\ &= -\eta \sum_{\tau=0}^{t-1} \beta^{t-1-\tau} \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_\tau) \\ &= -\eta \sum_{\tau=0}^{t-1} \beta^\tau \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_{t-1-\tau}). \end{aligned}$$

We define $\Gamma_{t-1} = \sum_{\tau=0}^{t-1} \beta^\tau = \frac{1-\beta^t}{1-\beta}$, so we know $\sum_{\tau=0}^{t-1} \frac{\beta^\tau}{\Gamma_{t-1}} = 1$ and we can apply Jensen's inequality to obtain

$$\begin{aligned} \|\theta_t - \theta_{t-1}\|^2 &= \left\| \eta \sum_{\tau=0}^{t-1} \frac{\beta^\tau}{\Gamma_{t-1}} \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_{t-1-\tau}) \right\|^2 \Gamma_{t-1}^2 \\ &\leq \eta^2 \Gamma_{t-1}^2 \sum_{\tau=0}^{t-1} \frac{\beta^\tau}{\Gamma_{t-1}} \left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_{t-1-\tau}) \right\|^2 \\ &= \eta^2 \Gamma_{t-1} \sum_{\tau=0}^{t-1} \beta^\tau \left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_{t-1-\tau}) \right\|^2 \end{aligned}$$

For any $\theta \in \mathbb{R}^d$, from Assumption 5.2, we have $\|\nabla \mathcal{L}^\mu(\theta_t)\| \leq G$. So using $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ and the result from Lemma 5.3, we have the following inequality holds for any θ_t

$$\begin{aligned} \mathbb{E}_{\mathcal{B}} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) \right\|^2 \right] &\leq 2\mathbb{E}_{\mathcal{B}} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}}(\theta_t) \right\|^2 \right] + 2\mathbb{E}_{\mathcal{B}} \left[\left\| \nabla \mathcal{L}_{\mathcal{B}}(\theta_t) \right\|^2 \right] \\ &\leq 2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2k}{\mu^2} + 4\epsilon^2 \end{aligned}$$

Combining with the Assumption 5.2 and $\Gamma_{t-1} = \sum_{\tau=0}^{t-1} \beta^\tau \leq \frac{1}{1-\beta}$, we have

$$\begin{aligned} \mathbb{E}_{\mathcal{B}_t} \left[\|\theta_t - \theta_{t-1}\|^2 \right] &\leq \eta^2 \Gamma_{t-1} \sum_{\tau=0}^{t-1} \beta^\tau \left(2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2k}{\mu^2} + 4\epsilon^2 \right) \\ &= \eta^2 \Gamma_{t-1}^2 \left(2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2k}{\mu^2} + 4\epsilon^2 \right) \\ &\leq \frac{\eta^2}{(1-\beta)^2} \left(2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2k}{\mu^2} + 4\epsilon^2 \right). \end{aligned}$$

Then we have

$$\begin{aligned}
 \|\nabla \mathcal{L}^\mu(m_t) - \nabla \mathcal{L}^\mu(\theta_t)\|^2 &= \mathbb{E}_{\mathcal{B}_t} \left[\|\nabla \mathcal{L}^\mu(m_t) - \nabla \mathcal{L}^\mu(\theta_t)\|^2 \right] \\
 &\leq \frac{L^2 \beta^2}{(1-\beta)^2} \mathbb{E}_{\mathcal{B}_t} \left[\|\theta_t - \theta_{t-1}\|^2 \right] \\
 &\leq \frac{L \beta^2 \eta^2}{(1-\beta)^4} \left(2G^2 + \frac{2\sigma^2}{B} + \frac{18M^2 k}{\mu^2} + 4\varepsilon^2 \right).
 \end{aligned}$$

This complete the proof. $\square$

Combining the result from Lemma 5.3 and M.1, we can complete the proof of Theorem 5.4.

Proof of Theorem 5.4. Because $\mu \geq \frac{6M}{\sqrt{\eta}}$ and $\varepsilon \leq \sqrt{\frac{\eta}{8}}$, then for any $\theta_t \in \mathbb{R}^d$, we have

$$\begin{aligned}
 \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] - \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 &\leq \mathbb{E}_{\mathcal{B}_t} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}_t}^\mu(\theta_t) \right\|^2 \right] \\
 &\leq \frac{9M^2 k}{\mu^2} + 2\varepsilon^2 \leq \frac{k+1}{4} \eta
 \end{aligned} \tag{15}$$

Additionally, the smoothed loss function $\mathcal{L}^\mu(\theta)$ is L -smooth respective to θ . Combining L -smoothness property of $\mathcal{L}^\mu(\theta)$ with the parameter update equation (6) under a mini-batch $\mathcal{B}_t$ and take the expectation on $\mathcal{B}_t$, we have

$$\begin{aligned}
 \mathbb{E}_{\mathcal{B}_t} [\mathcal{L}^\mu(m_{t+1})] &\leq \mathcal{L}^\mu(m_t) - \frac{\eta}{1-\beta} \nabla \mathcal{L}^\mu(m_t)^\top \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \\
 &\quad + \frac{L}{2} \mathbb{E}_{\mathcal{B}_t} \left[\left\| \frac{\eta}{1-\beta} \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right\|^2 \right].
 \end{aligned} \tag{16}$$

Besides, we have the following inequality for two vectors $a, b \in \mathbb{R}^d$

$$\begin{aligned}
 \|a + b\|^2 &\geq \|a\|^2 + \|b\|^2 - 2\|a\|\|b\| \geq \|b\|^2 - 2\|a\|\|b\| \\
 &\geq \|b\|^2 - 2\|a\|(\|b\|^2 + 1) = (1 - 2\|a\|)\|b\|^2 - 2\|a\|,
 \end{aligned} \tag{17}$$

then we have $-\|a + b\|^2 \leq (2\|a\| - 1)\|b\|^2 + 2\|a\|$. For the second term in the RHS of (16), we have

$$\begin{aligned}
 &\nabla \mathcal{L}^\mu(m_t)^\top \left(-\mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right) \\
 &= \left(\nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right) \left(-\mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right) - \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 \\
 &\stackrel{(a)}{\leq} \frac{1}{2} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 + \frac{1}{2} \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 - \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 \\
 &= \frac{1}{2} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 - \frac{1}{2} \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 \\
 &\stackrel{(b)}{\leq} \frac{1}{2} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 + \frac{1}{2} \left(2 \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] - \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 - 1 \right) \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 \\
 &\quad + \left\| \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] - \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 \\
 &\stackrel{(c)}{\leq} \frac{1}{2} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} \left[\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right] \right\|^2 + \frac{1}{2} \left(\frac{k+1}{2} \eta - 1 \right) \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{k+1}{4} \eta,
 \end{aligned} \tag{18}$$

where the inequality (a) is constructed with Young's inequality ($ab \leq \frac{a^p}{p} + \frac{b^q}{q}$ where $\frac{1}{p} + \frac{1}{q} = 1$), inequality (b) is constructed using (17) and $a = \mathbb{E}_{\mathcal{B}_t} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t)] - \nabla \mathcal{L}^\mu(\theta_t)$ and $b = \nabla \mathcal{L}^\mu(\theta_t)$, inequality (c) hold because of inequality

(15). Besides, by inequality $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$, and Assumption 5.2 (c), we have

$$\begin{aligned}
 \mathbb{E}_{\mathcal{B}_t} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) \right\|^2 \right] &\leq 2\mathbb{E}_{\mathcal{B}_t} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}_t}^\mu(\theta_t) \right\|^2 \right] + 2\mathbb{E}_{\mathcal{B}_t} \left[\left\| \nabla \mathcal{L}_{\mathcal{B}_t}^\mu(\theta_t) \right\|^2 \right] \\
 &\leq 2 \left(\left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{\sigma^2}{B} \right) + 2\mathbb{E}_{\mathcal{B}_t} \left[\left\| \hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t) - \nabla \mathcal{L}_{\mathcal{B}_t}^\mu(\theta_t) \right\|^2 \right] \\
 &\leq 2 \left(\left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{\sigma^2}{B} \right) + \frac{k+1}{2} \eta.
 \end{aligned} \tag{19}$$

Combining (16), (18), (19) and result in Lemma M.1, we have

$$\begin{aligned}
 &\mathbb{E}_{\mathcal{B}_t} [\mathcal{L}^\mu(m_{t+1})] - \mathcal{L}^\mu(m_t) \\
 &\leq \frac{\eta}{2(1-\beta)} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \mathbb{E}_{\mathcal{B}_t} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t)] \right\|^2 + \frac{\eta \left(\frac{k+1}{2} \eta - 1 \right)}{2(1-\beta)} \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{(k+1)\eta^2}{4(1-\beta)} \\
 &\quad + \frac{L\eta^2}{2(1-\beta)^2} \left(2\left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{2\sigma^2}{B} + \frac{k+1}{2} \eta \right) \\
 &\stackrel{(a)}{\leq} \frac{\eta}{1-\beta} \left\| \nabla \mathcal{L}^\mu(m_t)^\top - \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{\eta}{1-\beta} \left\| \nabla \mathcal{L}^\mu(\theta_t) - \mathbb{E}_{\mathcal{B}_t} [\hat{\nabla}^{\text{SZO}} \mathcal{L}_{\mathcal{B}_t}(\theta_t)] \right\|^2 + \frac{(k+1)\eta^2}{4(1-\beta)} \\
 &\quad + \frac{\eta \left(\frac{k+1}{2} \eta - 1 \right)}{2(1-\beta)} \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{L\eta^2}{2(1-\beta)^2} \left(2\left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{2\sigma^2}{B} + \frac{k+1}{2} \eta \right) \\
 &\stackrel{(b)}{\leq} \frac{L\beta^2\eta^3}{(1-\beta)^5} \left(2G^2 + \frac{2\sigma^2}{B} + \frac{k+1}{2} \eta \right) + \frac{(k+1)\eta^2}{4(1-\beta)} + \frac{(k+1)\eta^2}{4(1-\beta)} + \frac{\eta \left(\frac{k+1}{2} \eta - 1 \right)}{2(1-\beta)} \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 \\
 &\quad + \frac{L\eta^2}{2(1-\beta)^2} \left(2\left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{2\sigma^2}{B} + \frac{k+1}{2} \eta \right) \\
 &= \frac{2G^2L\beta^2\eta^3}{(1-\beta)^5} + \frac{2\sigma^2L\beta^2\eta^3}{(1-\beta)^5B} + \frac{(k+1)L\beta^2\eta^4}{2(1-\beta)^5} + \frac{(k+1)\eta^2}{2(1-\beta)} + \frac{\eta \left(\frac{k+1}{2} \eta - 1 \right)}{2(1-\beta)} \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 \\
 &\quad + \frac{L\eta^2}{(1-\beta)^2} \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{L\eta^2\sigma^2}{(1-\beta)^2B} + \frac{(k+1)L\eta^3}{4(1-\beta)^2} \\
 &= \left(\frac{L\eta^2}{(1-\beta)^2} + \frac{(k+1)\eta^2}{4(1-\beta)} - \frac{\eta}{2(1-\beta)} \right) \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \left(\frac{L\beta^2\eta^3}{(1-\beta)^5} + \frac{L\eta^2}{(1-\beta)^2} \right) \frac{\sigma^2}{B} \\
 &\quad + \frac{2G^2L\beta^2\eta^3}{(1-\beta)^5} + \frac{(k+1)L\beta^2\eta^4}{2(1-\beta)^5} + \frac{(k+1)\eta^2}{2(1-\beta)} + \frac{(k+1)L\eta^3}{4(1-\beta)^2} \\
 &\stackrel{(c)}{\leq} \left(\frac{(5+k)L\eta^2}{4(1-\beta)^2} - \frac{\eta}{2(1-\beta)} \right) \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 + \frac{L\eta^2(\eta+1)}{(1-\beta)^5} \cdot \frac{\sigma^2}{B} + \frac{2G^2L\beta^2\eta^3}{(1-\beta)^5} \\
 &\quad + \frac{(k+1)L\beta^2\eta^4}{2(1-\beta)^5} + \frac{(k+1)\eta^2}{2(1-\beta)} + \frac{(k+1)L\eta^3}{4(1-\beta)^2},
 \end{aligned}$$

where inequality (a) holds because of the relation $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$, the inequality (b) holds because of Lemma 5.3, Lemma M.1 and inequality (15), the inequality (c) holds because $L \geq 1$ and $\beta \in [0, 1)$. Re-arrange the above inequality, we have

$$\begin{aligned}
 \left(\frac{\eta}{2(1-\beta)} - \frac{(5+k)L\eta^2}{4(1-\beta)^2} \right) \left\| \nabla \mathcal{L}^\mu(\theta_t) \right\|^2 &\leq \mathcal{L}^\mu(m_t) - \mathbb{E}_{\mathcal{B}_t} [\mathcal{L}^\mu(m_{t+1})] + \frac{L\eta^2(\eta+1)}{(1-\beta)^5} \cdot \frac{\sigma^2}{B} \\
 &\quad + \frac{2G^2L\beta^2\eta^3}{(1-\beta)^5} + \frac{(k+1)L\beta^2\eta^4}{2(1-\beta)^5} \\
 &\quad + \frac{(k+1)\eta^2}{2(1-\beta)} + \frac{(k+1)L\eta^3}{4(1-\beta)^2}.
 \end{aligned}$$

Sum the above inequality from $t = 0$ to $T - 1$ with iterative expectation with $\mathcal{B}_0, \dots, \mathcal{B}_{t-1}$, we have

$$\begin{aligned} & \sum_{t=0}^{T-1} \left(\frac{\eta}{2(1-\beta)} - \frac{(5+k)L\eta^2}{4(1-\beta)^2} \right) \mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{t-1}} [\|\nabla \mathcal{L}^\mu(\theta_t)\|^2] \\ & \leq \sum_{t=0}^{T-1} [\mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{t-1}} [\mathcal{L}^\mu(m_t)] - \mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_t} [\mathcal{L}^\mu(m_{t+1})]] + T \left(\frac{L\eta^2(\eta+1)}{(1-\beta)^5} \cdot \frac{\sigma^2}{B} \right. \\ & \quad \left. + \frac{2G^2L\beta^2\eta^3}{(1-\beta)^5} + \frac{(k+1)L\beta^2\eta^4}{2(1-\beta)^5} + \frac{(k+1)\eta^2}{2(1-\beta)} + \frac{(k+1)L\eta^3}{4(1-\beta)^2} \right) \end{aligned}$$

Using $\mathcal{L}^\mu(\theta^*)$ to replace $\mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{T-1}} [\mathcal{L}^\mu(m_T)]$ because of $\mathcal{L}^\mu(\theta^*) \leq \mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{T-1}} [\mathcal{L}^\mu(m_T)]$, and $m_0 = \theta_0$, and given $\eta = \frac{1-\beta}{L\sqrt{T}}$ and $T \left(\frac{\eta}{2(1-\beta)} - \frac{(k+5)L\eta^2}{4(1-\beta)^2} \right) = \frac{\sqrt{T} - \frac{k+5}{2}}{2L}$, then the above inequality can be represented as

$$\begin{aligned} & \frac{1}{T} \sum_{t=0}^{T-1} \frac{\sqrt{T} - \frac{k+5}{2}}{2L} \mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{t-1}} [\|\nabla \mathcal{L}^\mu(\theta_t)\|^2] \\ & \leq \mathcal{L}^\mu(\theta_0) - \mathcal{L}^\mu(\theta^*) + T \left(\frac{L}{(1-\beta)^5} \cdot \frac{\sigma^2}{B} \cdot \frac{(1-\beta)^3}{L^3T\sqrt{T}} + \frac{L}{(1-\beta)^5} \cdot \frac{\sigma^2}{B} \cdot \frac{(1-\beta)^2}{L^2T} \right. \\ & \quad \left. + \frac{2G^2L\beta^2}{(1-\beta)^5} \cdot \frac{(1-\beta)^3}{L^3T\sqrt{T}} + \frac{(k+1)L\beta^2}{2(1-\beta)^5} \cdot \frac{(1-\beta)^4}{L^4T^2} \right. \\ & \quad \left. + \frac{(k+1)}{2(1-\beta)} \cdot \frac{(1-\beta)^2}{L^2T} + \frac{(k+1)L}{4(1-\beta)^2} \cdot \frac{(1-\beta)^3}{L^3T\sqrt{T}} \right) \\ & = \mathcal{L}^\mu(\theta_0) - \mathcal{L}^\mu(\theta^*) + \frac{1}{(1-\beta)^2L^2\sqrt{T}} \cdot \frac{\sigma^2}{B} + \frac{1}{L(1-\beta)^3} \cdot \frac{\sigma^2}{B} + \frac{2G^2\beta^2}{L^2(1-\beta)^2\sqrt{T}} + \frac{(k+1)\beta^2}{2L^3(1-\beta)T} \\ & \quad + \frac{(k+1)(1-\beta)}{2L^2} + \frac{(k+1)(1-\beta)}{4L^2\sqrt{T}} \end{aligned}$$

Without loss of generality, we choose $T > \left(\frac{k+5}{2} \right)^2$, then divide both side with $\frac{\sqrt{T} - \frac{k+5}{2}}{2L}$ we have

$$\begin{aligned} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}_{\mathcal{B}_0, \dots, \mathcal{B}_{t-1}} [\|\nabla \mathcal{L}^\mu(\theta_t)\|^2] & \leq \frac{2L [\mathcal{L}^\mu(\theta_0) - \mathcal{L}^\mu(\theta^*)]}{\sqrt{T} - \frac{k+5}{2}} + \frac{2}{(1-\beta)^2L\sqrt{T} \left(\sqrt{T} - \frac{k+5}{2} \right)} \cdot \frac{\sigma^2}{B} \\ & \quad + \frac{2}{(1-\beta)^3 \left(\sqrt{T} - \frac{k+5}{2} \right)} \cdot \frac{\sigma^2}{B} + \frac{4G^2\beta^2}{L(1-\beta)^2\sqrt{T} \left(\sqrt{T} - \frac{k+5}{2} \right)} \\ & \quad + \frac{(k+1)\beta^2}{L^2(1-\beta)T \left(\sqrt{T} - \frac{k+5}{2} \right)} + \frac{(k+1)(1-\beta)}{L \left(\sqrt{T} - \frac{k+5}{2} \right)} \\ & \quad + \frac{(k+1)(1-\beta)}{2L\sqrt{T} \left(\sqrt{T} - \frac{k+5}{2} \right)}. \end{aligned}$$

The convergence rate can be represented as the leading term, which is

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} [\|\nabla \mathcal{L}^\mu(\theta_t)\|^2] \leq \mathcal{O} \left(\frac{k(1-\beta)}{\sqrt{T}} + \left(\frac{1}{1-\beta} \right)^3 \frac{\sigma^2}{B\sqrt{T}} \right).$$

The proof is completed. $\square$